

Chapter 10. Signals

Introduction

Signals are software interrupts. They provide a way of handling asynchronous events. Most nontrivial application programs need to deal with signals.

POSIX reliable signals

Signals have been provided since the early versions of the UNIX System, but the signal model provided with systems such as Version 7 was not reliable. Signals could get lost, and it was difficult for a process to turn off selected signals when executing critical regions of code. Both 4.3BSD and SVR3 made changes to the signal model, adding what are called **reliable signals**. But the changes made by Berkeley and AT&T were incompatible. Fortunately, POSIX.1 standardized the reliable-signal routines, and that is what we describe here.

This chapter starts with an overview of signals and a description of what each signal is normally used for, then discusses problems with earlier implementations, since it is often important to understand what is wrong with an implementation before seeing how to do things correctly. This chapter contains numerous examples that are not entirely correct and a discussion of the defects.

Signal Concepts

- Every signal has a name. They all begin with the three characters `SIG`. For example:
 - `SIGABRT` is the abort signal that is generated when a process calls the `abort` function.
 - `SIGALRM` is the alarm signal that is generated when the timer set by the `alarm` function goes off.

FreeBSD 8.0 supports 32 different signals. Mac OS X 10.6.8 and Linux 3.2.0 each support 31 different signals, whereas Solaris 10 supports 40 different signals. FreeBSD, Linux, and Solaris, support additional application-defined signals introduced to support **real-time applications**.
- Signal names are all defined by positive integer constants (the signal number) in the header `<signal.h>`.
 - Implementations actually define the individual signals in a different header file, but this header file is included by `<signal.h>`.
 - It bad for the kernel to include header files meant for user-level applications, so if the applications and the kernel both need the same definitions, the information is placed in a kernel header file that is then included by the user-level header file.
 - `<sys/signal.h>` : FreeBSD 8.0 and Mac OS X 10.6.8
 - `<bits/signum.h>` : Linux 3.2.0
 - `<sys/iso/signal_iso.h>` : Solaris 10
- No signal has a signal number of 0. The `kill` function uses the signal number of 0 for a special case. POSIX.1 calls this value the null signal.

- Numerous conditions can generate a signal:
 - The terminal-generated signals occur when users press certain terminal keys. Pressing the DELETE key or Control-C on the terminal normally causes the interrupt signal (SIGINT) to be generated.
 - Hardware exceptions generate signals. For example, divide by 0 and invalid memory reference. These conditions are usually detected by the hardware, and the kernel is notified. The kernel then generates the appropriate signal for the process that was running at the time the condition occurred. For example, SIGSEGV is generated for a process that executes an invalid memory reference.
 - The kill(2) function allows a process to send any signal to another process or process group, with limitations: we have to be the owner of the process that we're sending the signal to, or we have to be the superuser.
 - The kill(1) command allows us to send signals to other processes. This program is just an interface to the kill function. This command is often used to terminate a runaway background process.
 - Software conditions can generate signals when a process should be notified of various events. For example:
 - SIGURG : generated when out-of-band data arrives over a network connection),
 - SIGPIPE : generated when a process writes to a pipe that has no reader)
 - SIGALRM : generated when an alarm clock set by the process expires).

Signals are classic examples of asynchronous events. They occur at random times to the process. The process can't simply test a variable (such as `errno`) to see whether a signal has occurred; instead, the process has to tell the kernel "if and when this signal occurs, do the following".

Signal dispositions

We can tell the kernel to do one of three things when a signal occurs. This is called the **disposition of the signal**, or the **action associated with a signal**. (`signal(7)` (<http://man7.org/linux/man-pages/man7/signal.7.html>))

1. **Ignore the signal.** Most signals can be ignored, but two signals can never be ignored: SIGKILL and SIGSTOP .
 - The reason these two signals can't be ignored is to provide the kernel and the superuser with a surefire way of either killing or stopping any process.
 - If we ignore some of the signals that are generated by a hardware exception (such as illegal memory reference or divide by 0), the behavior of the process is undefined.
2. **Catch the signal.** To do this, we tell the kernel to call a function of ours whenever the signal occurs. In our function, we can do whatever we want to handle the condition. For example:
 - If we're writing a command interpreter, when the user generates the interrupt signal at the keyboard, we probably want to return to the main loop of the program, terminating whatever command we were executing for the user.
 - If the SIGCHLD signal is caught, it means that a child process has terminated, so the signal-catching function can call `waitpid` to fetch the child's process ID and termination status.
 - If the process has created temporary files, we may want to write a signal-catching function for the SIGTERM signal (the termination signal that is the default signal sent by the kill command) to clean up the temporary files.

- Note that the two signals `SIGKILL` and `SIGSTOP` can't be caught.

3. Let the default action apply. Every signal has a default action. The default action for most signals is to terminate the process.

The signals `SIGKILL` and `SIGSTOP` cannot be caught, blocked, or ignored. (`signal(7)` (<http://man7.org/linux/man-pages/man7/signal.7.html>))

UNIX System signals

The following table lists the names of all the signals, an indication of which systems support the signal, and the default action for the signal. The SUS column contains "x" if the signal is defined as part of the base POSIX.1 specification and XSI if it is defined as part of the XSI option. The supported systems are FreeBSD 8.0, Linux 3.2.0, Mac OS X 10.6.8 and Solaris 10.

Name	Description	ISO		FreeBSD	Linux	Mac		Solaris	Default action
		C	SUS			OS X			
<code>SIGABRT</code>	abnormal termination (<code>abort</code>)	x	x	x	x	x		x	terminate+core
<code>SIGALRM</code>	timer expired (<code>alarm</code>)		x	x	x	x		x	terminate
<code>SIGBUS</code>	hardware fault		x	x	x	x		x	terminate+core
<code>SIGCANCEL</code>	threads library internal use							x	ignore
<code>SIGCHLD</code>	change in status of child		x	x	x	x		x	ignore
<code>SIGCONT</code>	continue stopped process		x	x	x	x		x	continue/ignore
<code>SIGEMT</code>	hardware fault			x	x	x		x	terminate+core
<code>SIGFPE</code>	arithmetic exception	x	x	x	x	x		x	terminate+core
<code>SIGFREEZE</code>	checkpoint freeze							x	ignore
<code>SIGHUP</code>	hangup		x	x	x	x		x	terminate
<code>SIGILL</code>	illegal instruction	x	x	x	x	x		x	terminate+core

Name	Description	ISO C	SUS	FreeBSD	Linux	Mac OS X	Solaris	Default action
SIGINFO	status request from keyboard			x		x		ignore
SIGINT	terminal interrupt character	x	x	x	x	x	x	terminate
SIGIO	asynchronous I/O			x	x	x	x	terminate/ignore
SIGIOT	hardware fault			x	x	x	x	terminate+core
SIGJVM1	Java virtual machine internal use						x	ignore
SIGJVM2	Java virtual machine internal use						x	ignore
SIGKILL	termination		x	x	x	x	x	terminate
SIGLOST	resource lost						x	terminate
SIGLWP	threads library internal use			x			x	ignore
SIGPIPE	write to pipe with no readers		x	x	x	x	x	terminate
SIGPOLL	pollable event (poll)				x		x	terminate
SIGPROF	profiling time alarm (setitimer)			x	x	x	x	terminate
SIGPWR	power fail/restart				x		x	terminate/ignore
SIGQUIT	terminal quit character		x	x	x	x	x	terminate+core
SIGSEGV	invalid memory reference	x	x	x	x	x	x	terminate+core

Name	Description	ISO C	SUS	FreeBSD	Linux	Mac OS X	Solaris	Default action
SIGSTKFLT	coprocessor stack fault				x			terminate
SIGSTOP	stop		x	x	x	x	x	stop process
SIGSYS	invalid system call		XSI	x	x	x	x	terminate+core
SIGTERM	termination	x	x	x	x	x	x	terminate
SIGTHAW	checkpoint thaw						x	ignore
SIGTHR	threads library internal use			x			x	terminate
SIGTRAP	hardware fault		XSI	x	x	x	x	terminate+core
SIGTSTP	terminal stop character		x	x	x	x	x	stop process
SIGTTIN	background read from control tty		x	x	x	x	x	stop process
SIGTTOU	background write to control tty		x	x	x	x	x	stop process
SIGURG	urgent condition (sockets)		x	x	x	x	x	ignore
SIGUSR1	user-defined signal		x	x	x	x	x	terminate
SIGUSR2	user-defined signal		x	x	x	x	x	terminate
SIGVTALRM	virtual time alarm (setitimer)		XSI	x	x	x	x	terminate
SIGWAITING	threads library internal use						x	ignore
SIGWINCH	terminal window size change			x	x	x	x	ignore

Name	Description	ISO		FreeBSD	Linux	Mac		Default action
		C	SUS			OS X	Solaris	
SIGXCPU	CPU limit exceeded (<code>setrlimit</code>)		XSI	x	x	x	x	terminate+core/ignore
SIGXFSZ	file size limit exceeded (<code>setrlimit</code>)		XSI	x	x	x	x	terminate+core/ignore
SIGXRES	resource control exceeded						x	ignore

The core file

When the default action (in the table above) is labeled "terminate+core", it means that a memory image of the process is left in the file named `core` of the current working directory of the process. This file can be used with most UNIX System debuggers to examine the state of the process at the time it terminated.

The name of the `core` file varies among implementations. On Mac OS X 10.6.8, the core file is named `core.pid`, where `pid` is the ID of the process that received the signal. On Linux 3.2.0, the name is configured through `/proc/sys/kernel/core_pattern`. (`core(5)` (<http://man7.org/linux/man-pages/man5/core.5.html>)) [p315]

Most implementations leave the core file in the current working directory of the corresponding process; Mac OS X places all core files in `/cores` instead.

The core file will not be generated if:

- the process was set-user-ID and the current user is not the owner of the program file,
- the process was set-group-ID and the current user is not the group owner of the file,
- the user does not have permission to write in the current working directory,
- the file already exists and the user does not have permission to write to it,
- the file is too big (see `RLIMIT_CORE` limit in Section 7.11 ([../ch7/#getrlimit-and-setrlimit-functions](#)))

The permissions of the core file (assuming that the file doesn't already exist) are usually user-read and user-write, although Mac OS X sets only user-read.

In the table, the signals with a description of "hardware fault" correspond to implementation-defined hardware faults.

Detailed description of signals

- **SIGABRT** : generated by calling the `abort` function. The process terminates abnormally.
- **SIGALRM** :
 - This signal is generated when a timer set with the `alarm` function expires.
 - This signal is also generated when an interval timer set by the `setitimer(2)` function expires.

- **SIGBUS** : indicates an implementation-defined hardware fault. Implementations usually generate this signal on certain types of memory faults.
- **SIGCANCEL** : used internally by the Solaris threads library. It is not meant for general use.
- **SIGCHLD** : Whenever a process terminates or stops, the **SIGCHLD** signal is sent to the parent. By default, this signal is ignored, so the parent must catch this signal if it wants to be notified whenever a child's status changes. The normal action in the signal-catching function is to call one of the **wait** functions to fetch the child's process ID and termination status. [p317]
- **SIGCONT** : this job-control signal is sent to a stopped process when it is continued. The default action is to continue a stopped process, but to ignore the signal if the process wasn't stopped.
- **SIGEMT** : indicates an implementation-defined hardware fault. Not all platforms support this signal. [p318]
- **SIGFPE** : signals an arithmetic exception, such as divide by 0, floating-point overflow, and so on. The name is derived from "floating-point exception" (Program Error Signals (http://www.gnu.org/software/libc/manual/html_node/Program-Error-Signals.html)).
- **SIGFREEZE** : defined only by Solaris. [p318]
- **SIGHUP** : this signal is sent to the controlling process (session leader) associated with a controlling terminal if a disconnect is detected by the terminal interface.
 - This signal is generated for this condition only if the terminal's **CLOCAL** flag is not set. The **CLOCAL** flag for a terminal is set if the attached terminal is local. The flag tells the terminal driver to ignore all modem status lines.
 - The session leader that receives this signal may be in the background (Figure 9.7 ([../figure_9.7.png](#))). This differs from the normal terminal-generated signals (interrupt, quit, and suspend), which are always delivered to the foreground process group.
 - This signal is also generated if the session leader terminates. In this case, the signal is sent to each process in the foreground process group.
 - This signal is commonly used to notify daemon processes (Chapter 13 ([../ch13/](#))) to reread their configuration files. The reason **SIGHUP** is chosen for this task is that a daemon should not have a controlling terminal and would normally never receive this signal.
- **SIGILL** : indicates that the process has executed an illegal hardware instruction.
 - 4.3BSD generated this signal from the **abort** function. **SIGABRT** is now used for this purpose.
- **SIGINFO** : This BSD signal is generated by the terminal driver when we type the status key (often Control-T). This signal is sent to all processes in the foreground process group (Figure 9.9). This signal normally causes status information on processes in the foreground process group to be displayed on the terminal. Linux doesn't provide support for **SIGINFO** .
- **SIGINT** : generated by the terminal driver when we press the interrupt key (often DELETE or Control-C). This signal is sent to all processes in the foreground process group (Figure 9.9 ([../figure_9.9.png](#))). This signal is often used to terminate a runaway program, especially when it's generating a lot of unwanted output on the screen.
- **SIGIO** : indicates an asynchronous I/O event.
- **SIGIOT** : indicates an implementation-defined hardware fault. **SIGABRT** is now used for this purpose. On FreeBSD 8.0, Linux 3.2.0, Mac OS X 10.6.8, and Solaris 10, **SIGIOT** is defined to be the same value as **SIGABRT** .

- `SIGJVM1` and `SIGJVM2` : reserved for use by the Java virtual machine on Solaris.
- `SIGKILL` : one of the two that can't be caught or ignored. It provides the system administrator with a sure way to kill any process.
- `SIGLOST` : used to notify a process running on a Solaris NFSv4 client system that a lock could not be reacquired during recovery.
- `SIGPIPE` : If we write to a pipeline but the reader has terminated, `SIGPIPE` is generated. This signal is also generated when a process writes to a socket of type `SOCK_STREAM` that is no longer connected.
- `SIGPOLL` : This signal is marked obsolescent in SUSv4, so it might be removed in a future version of the standard. It can be generated when a specific event occurs on a pollable device.
- `SIGPROF` : This signal is marked obsolescent in SUSv4, so it might be removed in a future version of the standard. This signal is generated when a profiling interval timer set by the `setitimer(2)` function expires.
- `SIGPWR` : system dependent, mainly used on a system that has an uninterruptible power supply (UPS).
 - If power fails, the UPS takes over and the software can usually be notified. Nothing needs to be done at this point, as the system continues running on battery power. But if the battery gets low, the software is usually notified again; at this point, it behooves the system to shut everything down. The process that is notified of the low-battery condition sends the `SIGPWR` signal to the `init` process, and `init` handles the system shutdown.
 - Solaris 10 and some Linux distributions have entries in the `inittab` file for this purpose: `powerfail` and `powerwait` (or `powerokwait`).
 - The default action for `SIGPWR` is either "terminate" or "ignore", which depends on the system. The default on Linux is to terminate the process. On Solaris, the signal is ignored by default.
- `SIGQUIT` : generated by the terminal driver when we press the terminal quit key (often Control-backslash). This signal is sent to all processes in the foreground process group (Figure 9.9 (../figure_9.9.png)). This signal not only terminates the foreground process group (as does `SIGINT`), but also generates a `core` file.
- `SIGSEGV` : indicates that the process has made an invalid memory reference (which is usually a sign that the program has a bug, such as dereferencing an uninitialized pointer). The name `SEGV` stands for "segmentation violation".
- `SIGSTKFLT` : This signal is defined only by Linux. It showed up in the earliest versions of Linux, where it was intended to be used for stack faults taken by the math coprocessor. This signal is not generated by the kernel, but remains for backward compatibility.
- `SIGSTOP` : This job-control signal stops a process. It is similar to the interactive stop signal (`SIGTSTP`), but `SIGSTOP` cannot be caught or ignored.
- `SIGSYS` : indicates an invalid system call. The process executed a machine instruction that the kernel thought was a system call, but the parameter with the instruction that indicates the type of system call was invalid. For example, if you build a program that uses a new system call and you then try to run the same binary on an older version of the operating system where the system call doesn't exist. [p320]
- `SIGTERM` : the termination signal sent by the `kill(1)` command by default. Because it can be caught by applications, using `SIGTERM` gives programs a chance to terminate gracefully by cleaning up before exiting (in contrast to `SIGKILL`, which can't be caught or ignored).
- `SIGTHAW` : defined only by Solaris and used to notify processes that need to take special action when the system resumes operation after being suspended.

- **SIGTHR** : reserved for use by the thread library on FreeBSD. It is defined to have the same value as **SIGLWP** .
- **SIGTRAP** : indicates an implementation-defined hardware fault. The signal name comes from the PDP-11 TRAP instruction. Implementations often use this signal to transfer control to a debugger when a breakpoint instruction is executed.
- **SIGTSTP** : This interactive stop signal is generated by the terminal driver when we press the terminal suspend key (often Control-Z). This signal is sent to all processes in the foreground process group (Figure 9.9 ([../figure_9.9.png](#))). [p321]
- **SIGTTIN** : generated by the terminal driver when a process in a background process group tries to read from its controlling terminal. If either of the following case occurs, the signal is not generated; instead, the read operation fails with `errno` set to `EIO` :
 - The reading process is ignoring or blocking this signal.
 - The process group of the reading process is orphaned.
- **SIGTTOU** : generated by the terminal driver when a process in a background process group tries to write to its controlling terminal. Unlike the case with background reads, a process can choose to allow background writes to the controlling terminal. If background writes are not allowed, then like the **SIGTTIN** signal, the signal is not generated if either of the following cases occurs; instead, the read operation fails with `errno` set to `EIO` :
 - The writing process is ignoring or blocking this signal
 - The process group of the writing process is orphaned

Regardless of whether background writes are allowed, certain terminal operations (other than writing), including `tcsetattr` , `tcsendbreak` , `tcdrain` , `tcflush` , `tcflow` , and `tcsetpgrp` can also generate the **SIGTTOU** signal.
- **SIGURG** : notifies the process that an urgent condition has occurred. It is optionally generated when out-of-band data (https://en.wikipedia.org/wiki/Out-of-band_data) is received on a network connection.
- **SIGUSR1** and **SIGUSR2** : user-defined signals, for use in application programs.
- **SIGVTALRM** : generated when a virtual interval timer set by the `setitimer(2)` function expires.
- **SIGWAITING** : used internally by the Solaris threads library, and is not available for general use.
- **SIGWINCH** : The kernel maintains the size of the window associated with each terminal and pseudo terminal. A process can get and set the window size with the `ioctl` function. If a process changes the window size from its previous value using the `ioctl` set-window-size command, the kernel generates the **SIGWINCH** signal for the foreground process group.
- **SIGXCPU** : generated if the process exceeds its soft CPU time limit. The default action depends on the operating system. The Single UNIX Specification requires that the default action be to terminate the process abnormally.
 - Linux 3.2.0 and Solaris 10 support a default action of terminate with a core file
 - FreeBSD 8.0 and Mac OS X 10.6.8 support a default action of terminate without generating a core file.
- **SIGXFSZ** : generated if the process exceeds its soft file size limit. The default action depends on the operating system, similar to **SIGXCPU** .
- **SIGXRES** : defined only by Solaris.

signal Function

The simplest interface to the signal features of the UNIX System is the `signal` function

`apue_signal.h` (<https://gist.github.com/shichao-an/73f4115f93c26a852349>)

```
#include <signal.h>

void (*signal(int signo, void (*func)(int)))(int);

/* Returns: previous disposition of signal (see following) if OK, SIG_ERR on error
 */
```

Implementations derived from UNIX System V support the `signal` function, which provides the old unreliable-signal semantics. New applications should not use these unreliable signals. 4.4BSD also provides the `signal` function, but it is defined in terms of the `sigaction` function, so using it under 4.4BSD provides the newer reliable-signal semantics. Most current systems follow this strategy except Solaris. [p323]

Because the semantics of signal differ among implementations, we must use the `sigaction` function instead. We provide an implementation of `signal` that uses `sigaction` (later this chapter).

Arguments:

- The *signo* argument is the name of the signal from previous table.
- The value of *func* one of the following:
 - the constant `SIG_IGN` , which tells the system ignore the signal;
 - the constant `SIG_DFL` , which sets the action associated with the signal to its default value;
 - the address of a function to be called when the signal occurs, which arranges to "catch" the signal. This function is called either the **signal handler** or the **signal-catching function**.

The prototype for the `signal` function states that the function requires two arguments and returns a pointer to a function that returns nothing (`void`):

- The first argument, *signo*, is an integer.
- The second argument, *func*, is a pointer to a function that takes a single integer argument and returns nothing.
- The returned function (function whose address is returned as the value of `signal`) takes a single integer argument (the final (`int`)).

In plain English, this declaration says that the signal handler is passed a single integer argument (the signal number) and that it returns nothing. When we call `signal` to establish the signal handler, the second argument is a pointer to the function. The return value from `signal` is the pointer to the previous signal handler.

The `signal` function prototype can be made much simpler through the use of the following `typedef` :

```
typedef void Sigfunc(int);
```

Then the prototype becomes:

```
Sigfunc *signal(int, Sigfunc *);
```

This `typedef` is included in `apue.h` (<https://github.com/shichao-an/apue.3e/blob/master/include/apue.h#L41>) and is used with the functions in this chapter.

If we examine the system's header `<signal.h>`, we will probably find declarations of the form:

```
#define SIG_ERR (void (*)())-1  
#define SIG_DFL (void (*)())0  
#define SIG_IGN (void (*)())1
```

These constants can be used in place of the "pointer to a function that takes an integer argument and returns nothing", the second argument to `signal`, and the return value from `signal`. The three values used for these constants need not be `-1`, `0`, and `1`. They must be three values that can never be the address of any declarable function. Most UNIX systems use the values shown. (See [Doubts and Solutions](#) for details)

Example:

The following code shows a simple signal handler that catches either of the two user-defined signals and prints the signal number.

- `sigusr.c` (<https://github.com/shichao-an/apue.3e/blob/master/signals/sigusr.c>)

```

#include "apue.h"

static void sig_usr(int); /* one handler for both signals */

int
main(void)
{
    if (signal(SIGUSR1, sig_usr) == SIG_ERR)
        err_sys("can't catch SIGUSR1");
    if (signal(SIGUSR2, sig_usr) == SIG_ERR)
        err_sys("can't catch SIGUSR2");
    for ( ; ; )
        pause();
}

static void
sig_usr(int signo) /* argument is signal number */
{
    if (signo == SIGUSR1)
        printf("received SIGUSR1\n");
    else if (signo == SIGUSR2)
        printf("received SIGUSR2\n");
    else
        err_dump("received signal %d\n", signo);
}

```

We invoke the program in the background and use the `kill(1)` command to send it signals. The term *kill* in the UNIX System is a misnomer. The `kill(1)` command and the `kill(2)` function just send a signal to a process or process group. Whether that signal terminates the process depends on which signal is sent and whether the process has arranged to catch the signal.

Result:

```

$ ./a.out &                # start process in background
[1] 7216                    # job-control shell prints job number and process ID
$ kill -USR1 7216           # send it SIGUSR1
received SIGUSR1
$ kill -USR2 7216          # send it SIGUSR2
received SIGUSR2
$ kill 7216                 # now send it SIGTERM
[1]+ Terminated ./a.out

```

When we send the `SIGTERM` signal, the process is terminated, since it doesn't catch the signal, and the default action for the signal is termination.

Program Start-Up

When a program is executed, the status of all signals is either default or ignore. All signals are set to their default action, unless the process that calls `exec` is ignoring the signal. The `exec` functions change the disposition of any signals being caught to their default action and leave the status of all other signals alone. The reason is that a signal that is being caught by a process that calls `exec` cannot be caught by the same function in the new program, since the address of the signal-catching function in the caller probably has no meaning in the new program file that is executed. [p325]

With a shell that doesn't support job control, when we execute a process in the background:

```
cc main.c &
```

The shell automatically sets the disposition of the interrupt and quit signals in the background process to be ignored. This is done so that if we type the interrupt character, it doesn't affect the background process. If this weren't done and we typed the interrupt character, it would terminate not only the foreground process, but also all the background processes.

Many interactive programs that catch these two signals have code that looks like:

```
void sig_int(int), sig_quit(int);

if (signal(SIGINT, SIG_IGN) != SIG_IGN)
    signal(SIGINT, sig_int);
if (signal(SIGQUIT, SIG_IGN) != SIG_IGN)
    signal(SIGQUIT, sig_quit);
```

Following this approach, the process catches the signal only if the signal is not currently being ignored.

The `signal` function has a limitation: we are not able to determine the current disposition of a signal without changing the disposition. The `sigaction` function (discussed later in this chapter) allows us to determine a signal's disposition without changing it.

Process Creation

When a process calls `fork`, the child inherits the parent's signal dispositions. Here, since the child starts off with a copy of the parent's memory image, the address of a signal-catching function has meaning in the child.

Unreliable Signals

In earlier versions of the UNIX System, signals were unreliable, which means that signals could get lost: a signal could occur and the process would never know about it. [p326]

One problem with these early versions was that the action for a signal was reset to its default each time the signal occurred. The code that was described usually looked like:

```

    int sig_int(); /* my signal handling function */
    ...
    signal(SIGINT, sig_int); /* establish handler */
    ...

sig_int()
{
    signal(SIGINT, sig_int); /* reestablish handler for next time */
    ... /* process the signal ... */
}

```

The problem with this code fragment is that there is a window of time (after the signal has occurred, but before the call to `signal` in the signal handler) when the interrupt signal could occur another time. This second signal would cause the default action to occur, which terminates the process. This is one of those conditions that works correctly most of the time, causing us to think that it is correct, when it isn't.

Another problem with these earlier systems was that the process was unable to turn a signal off when it didn't want the signal to occur. All the process could do was ignore the signal. There are times when we would like to tell the system "prevent the following signals from interrupting me, but remember if they do occur". The following code catches a signal and sets a flag for the process that indicates that the signal occurred:

```

int sig_int(); /* my signal handling function */
int sig_int_flag; /* set nonzero when signal occurs */

main()
{
    signal(SIGINT, sig_int); /* establish handler */
    ...
    while (sig_int_flag == 0)
        pause(); /* go to sleep, waiting for signal */
    ...
}

sig_int()
{
    signal(SIGINT, sig_int); /* reestablish handler for next time */
    sig_int_flag = 1; /* set flag for main loop to examine */
}

```

The process is calling the `pause` function to put it to sleep until a signal is caught. When the signal is caught, the signal handler just sets the flag `sig_int_flag` to a nonzero value. The process is automatically awakened by the kernel after the signal handler returns, notices that the flag is nonzero, and does whatever it needs to do. But there

is a window of time when things can go wrong. If the signal occurs after the test of `sig_int_flag` but before the call to pause, the process could go to sleep forever (assuming that the signal is never generated again). This occurrence of the signal is lost.

Interrupted System Calls

In earlier UNIX systems, if a process caught a signal while the process was blocked in a "slow" system call, the system call was interrupted. The system call returned an error and `errno` was set to `EINTR`. This was done under the assumption that since a signal occurred and the process caught it, there is a good chance that something has happened that should wake up the blocked system call.

Slow system calls

The system calls are divided into two categories: the "slow" system calls and all the others. The slow system calls are those that can block forever:

- Reads that can block the caller forever if data isn't present with certain file types (pipes, terminal devices, and network devices)
- Writes that can block the caller forever if the data can't be accepted immediately by these same file types
- Opens on certain file types that block the caller until some condition occurs (such as a terminal device open waiting until an attached modem answers the phone)
- The `pause` function (which by definition puts the calling process to sleep until a signal is caught) and the `wait` function
- Certain `ioctl` operations
- Some of the interprocess communication functions

The notable exception to these slow system calls is anything related to disk I/O. Although a read or a write of a disk file can block the caller temporarily (while the disk driver queues the request and then the request is executed), unless a hardware error occurs, the I/O operation always returns and unblocks the caller quickly.

Historically, POSIX.1 semantics gave implementations a choice of how to deal with `reads` and `writes` that have processed partial amounts of data, implementations derived from System V fail the system call, whereas BSD-derived implementations return partial success. With the 2001 version of the POSIX.1 standard, the BSD-style semantics are required. [p328]

The problem with interrupted system calls is that we now have to handle the error return explicitly. Assuming a read operation and assuming that we want to restart the read even if it's interrupted, the typical code sequence would be:

```
again:
    if ((n = read(fd, buf, BUFSIZE)) < 0) {
        if (errno == EINTR)
            goto again; /* just an interrupted system call */
        /* handle other errors */
    }
```

Automatic restarts of interrupted system calls

Automatic restarting of certain interrupted system calls were introduced since 4.2BSD to prevent applications from having to handle interrupted system calls. The system calls that were automatically restarted are:

- Functions that are interrupted by a signal only if they are operating on a slow device:
 - `ioctl`
 - `read`
 - `readv`
 - `write`
 - `writew`
- Functions that are always interrupted when a signal is caught.
 - `wait`
 - `waitpid`

Some applications didn't want the operation restarted if it was interrupted; 4.3BSD allowed the process to disable this feature on a per-signal basis.

Difference between the `signal` and `sigaction` functions on restarts

POSIX.1 requires an implementation to restart system calls only when the `SA_RESTART` flag is in effect for the interrupting signal. This flag is used with the `sigaction` function to allow applications to request that interrupted system calls be restarted.

Historically, when using the `signal` function to establish a signal handler, implementations varied with respect to how interrupted system calls were handled. System V never restarted system calls by default. BSD, in contrast, restarted them if the calls were interrupted by signals. On FreeBSD 8.0, Linux 3.2.0, and Mac OS X 10.6.8, when signal handlers are installed with the `signal` function, interrupted system calls will be restarted. By using our own implementation of the `signal` function, we avoid having to deal with these differences. [p329]

One reason 4.2BSD introduced the automatic restart feature is that sometimes we don't know that the input or output device is a slow device. [p329]

The figure below summarizes the signal functions and their semantics provided by the various implementations.

Functions	System	Signal handler remains installed	Ability to block signals	Automatic restart of interrupted system calls?
signal	ISO C, POSIX.1	unspecified	unspecified	unspecified
	V7, SVR2, SVR3			never
	SVR4, Solaris			never
	4.2BSD	•	•	always
	4.3BSD, 4.4BSD, FreeBSD, Linux, Mac OS X	•	•	default
sigaction	POSIX.1, 4.4BSD, SVR4, FreeBSD, Linux, Mac OS X, Solaris	•	•	optional

(../figure_10.3.png)

Later this chapter, we provide our own version of the `signal` function that automatically tries to restart interrupted system calls (other than for the `SIGALRM` signal), and `signal_intr`, that tries to never do the restart.

Reentrant Functions

When a signal that is being caught is handled by a process, the normal sequence of instructions being executed by the process is temporarily interrupted by the signal handler. The process then continues executing, but the instructions in the signal handler are now executed. If the signal handler returns (instead of calling `exit` or `longjmp`), then the normal sequence of instructions that the process was executing when the signal was caught continues executing. This is similar to what happens when a hardware interrupt occurs.

However, in the signal handler, we can't tell where the process was executing when the signal was caught:

- What if the process was in the middle of allocating additional memory on its heap using `malloc`, and we call `malloc` from the signal handler?
 - Havoc can result for the process, since `malloc` usually maintains a linked list of all its allocated areas, and it may have been in the middle of changing this list.
- What if the process was in the middle of a call to a function, such as `getpwnam` (Section 6.2 (../ch6/#password-file)), that stores its result in a static location, and we call the same function from the signal handler?
 - The information returned to the normal caller can get overwritten with the information returned to the signal handler.

The Single UNIX Specification specifies the functions that are guaranteed to be safe to call from within a signal handler. These functions are reentrant and are called **async-signal safe** by the SUS. Besides being reentrant, they block any signals during operation if delivery of a signal might cause inconsistencies.

The following table lists these async-signal safe functions, which are reentrant functions that may be called from a signal handler.

abort	faccessat	linkat	select	socketpair
accept	fchmod	listen	sem_post	stat
access	fchmodat	lseek	send	symlink
aio_error	fchown	lstat	sendmsg	symlinkat
aio_return	fchownat	mkdir	sendto	tcdrain
aio_suspend	fcntl	mkdirat	setgid	tcflow
alarm	fdatasync	mkfifo	setpgid	tcflush
bind	fexecve	mkfifoat	setsid	tcgetattr
cfgetispeed	fork	mknod	setsockopt	tcgetpgrp
cfgetospeed	fstat	mknodat	setuid	tcsendbreak
cfsetispeed	fstatat	open	shutdown	tcsetattr
cfsetospeed	fsync	openat	sigaction	tcsetpgrp
chdir	ftruncate	pause	sigaddset	time
chmod	futimens	pipe	sigdelset	timer_getoverrun
chown	getegid	poll	sigemptyset	timer_gettime
clock_gettime	geteuid	posix_trace_event	sigfillset	timer_settime
close	getgid	pselect	sigismember	times
connect	getgroups	raise	signal	umask
creat	getpeername	read	sigpause	uname
dup	getpgrp	readlink	sigpending	unlink
dup2	getpid	readlinkat	sigprocmask	unlinkat
exec1	getppid	recv	sigqueue	utime

<code>execl</code>	<code>getsockname</code>	<code>recvfrom</code>	<code>sigset</code>	<code>utimensat</code>
<code>execv</code>	<code>getsockopt</code>	<code>recvmsg</code>	<code>sigsuspend</code>	<code>utimes</code>
<code>execve</code>	<code>getuid</code>	<code>rename</code>	<code>sleep</code>	<code>wait</code>
<code>_Exit</code>	<code>kill</code>	<code>renameat</code>	<code>socketatmark</code>	<code>waitpid</code>
<code>_exit</code>	<code>link</code>	<code>rmdir</code>	<code>socket</code>	<code>write</code>

Most of the functions that are not included in table above are missing because:

- They are known to use static data structures;
- They call `malloc` or `free` ;
- They are part of the standard I/O library.

Note when using the functions in the table above:

- Most implementations of the standard I/O library use global data structures in a nonreentrant way.
- Be aware that even if we call a function listed in the table above from a signal handler, there is only one `errno` variable per thread (recall the discussion of `errno` and threads in Section 1.7 (../ch1/#error-handling)), and we might potentially modify its value.
 - Consider a signal handler that is invoked right after `main` has set `errno` . If the signal handler calls `read` , for example, this call can change the value of `errno` , wiping out the value that was just stored in `main` .

Therefore, as a general rule, when calling the functions listed in the table above from a signal handler, we should save and restore `errno` . Be aware that a commonly caught signal is `SIGCHLD` , and its signal handler usually calls one of the wait functions. All the `wait` functions can change `errno` .

- `longjmp` (Section 7.10 (../ch7/#setjmp-and-longjmp-functions)) and `siglongjmp` (Section 10.15) are missing from the table because the signal may have occurred while the `main` routine was updating a data structure in a nonreentrant way. This data structure could be left half updated if we call `siglongjmp` instead of returning from the signal handler. If it is going to do such things as update global data structures, while catching signals that cause `sigsetjmp` to be executed, an application needs to block the signals while updating the data structures.

Example of calling `getpwnam` from a signal handler *

The code below shows a program that calls the nonreentrant function `getpwnam` from a signal handler that is called every second. We use the `alarm` function (Section 10.10 (../alarm-and-pause-functions)) here to generate a `SIGALRM` signal every second.

`signals/reenter.c` (<https://github.com/shichao-an/apue.3e/blob/master/signals/reenter.c>)

```

#include "apue.h"
#include <pwd.h>

static void
my_alarm(int signo)
{
    struct passwd    *rootptr;

    printf("in signal handler\n");
    if ((rootptr = getpwnam("root")) == NULL)
        err_sys("getpwnam(root) error");
    alarm(1);
}

int
main(void)
{
    struct passwd    *ptr;

    signal(SIGALRM, my_alarm);
    alarm(1);
    for ( ; ; ) {
        if ((ptr = getpwnam("sar")) == NULL)
            err_sys("getpwnam error");
        if (strcmp(ptr->pw_name, "sar") != 0)
            printf("return value corrupted!, pw_name = %s\n",
                ptr->pw_name);
    }
}

```

When this program was run, the results were random. Usually, the program would be terminated by a `SIGSEGV` signal when the signal handler returned after several iterations. An examination of the `core` file showed that the main function had called `getpwnam`, but that when `getpwnam` called `free`, the signal handler interrupted it and called `getpwnam`, which in turn called `free`. The data structures maintained by `malloc` and `free` had been corrupted when the signal handler (indirectly) called `free` while the main function was also calling `free`. Occasionally, the program would run for several seconds before crashing with a `SIGSEGV` error. When the main function did run correctly after the signal had been caught, the return value was sometimes corrupted and sometimes fine.

SIGCLD Semantics

Two signals that continually generate confusion are `SIGCLD` and `SIGCHLD`. The name `SIGCLD` (without the `H`) is from System V, and this signal has different semantics from the BSD signal, named `SIGCHLD`. The POSIX.1 signal is also named `SIGCHLD`.

The semantics of the BSD `SIGCHLD` signal are normal and its semantics are similar to all other signals. When the signal occurs, the status of a child has changed, and we need to call one of the `wait` functions to determine what has happened.

System V, however, has traditionally handled the `SIGCLD` signal differently from other signals:

1. If the process specifically sets its disposition to `SIG_IGN`, children of the calling process will not generate zombie processes. [p333]
 - 4.4BSD always generates zombies if `SIGCHLD` is ignored. If we want to avoid zombies, we have to `wait` for our children.
2. If we set the disposition of `SIGCLD` to be caught, the kernel immediately checks whether any child processes are ready to be `wait` ed for and, if so, calls the `SIGCLD` handler. [p333-335]
 - FreeBSD 8.0 and Mac OS X 10.6.8 don't exhibit this problem, because BSD-based systems generally don't support historical System V semantics for `SIGCLD`.
 - Linux 3.2.0 also doesn't exhibit this problem, because it doesn't call the `SIGCHLD` signal handler when a process arranges to catch `SIGCHLD` and child processes are ready to be waited for, even though `SIGCLD` and `SIGCHLD` are defined to be the same value.
 - Solaris avoids this problem by including extra code in the kernel.

Of the four platforms described in this text, only Linux 3.2.0 and Solaris 10 define `SIGCLD`. On these platforms, `SIGCLD` is equivalent to `SIGCHLD`.

Reliable-Signal Terminology and Semantics

This section defines some terms used through the discussion of signals.

- A signal is *generated* for a process (or sent to a process) when the event that causes the signal occurs. When the signal is generated, the kernel usually sets a flag of some form in the process table. The event could be:
 - Hardware exception (e.g., divide by 0),
 - Software condition (e.g., an alarm timer expiring),
 - Terminal-generated signal,
 - A call to the `kill` function.
- A signal is *delivered* to a process when the action for a signal is taken.
- A signal is *pending* during the time between its generation and delivery.
- A process has the option of *blocking* the delivery of a signal. If a signal that is blocked is generated for a process, and if the action for that signal is either the default action or to catch the signal, then the signal remains pending for the process until the process either:
 - unblocks the signal, or
 - changes the action to ignore the signal.

The system determines what to do with a blocked signal when the signal is delivered, not when it's generated. This allows the process to change the action for the signal before it's delivered. The `sigpending` function (Section 10.13) can be called by a process to determine which signals are blocked and pending.

POSIX.1 allows the system to deliver the signal either once or more than once in case a blocked signal is generated more than once before the process unblocks the signal. If the system delivers the signal more than once, we say that the signals are *queued*. Most UNIX systems, however, do not queue signals unless they support the real-time extensions to POSIX.1. Instead, the UNIX kernel simply delivers the signal once. [p336]

Section 10.20 discusses queueing signals further.

POSIX.1 does not specify the order in which the signals are delivered to the process. The Rationale for POSIX.1 does suggest, however, that signals related to the current state of the process be delivered before other signals. (`SIGSEGV` is one such signal.)

Each process has a **signal mask** that defines the set of signals currently blocked from delivery to that process. This mask has one bit for each possible signal. If the bit is on for a given signal, that signal is currently blocked. A process can examine and change its current signal mask by calling `sigprocmask` (Section 10.12). Since it is possible for the number of signals to exceed the number of bits in an integer, POSIX.1 defines a data type, called `sigset_t`, that holds a **signal set**. The signal mask is stored in one of these signal sets. The five functions that operate on signal sets are described in Section 10.11.

kill and raise Functions

- The `kill` function sends a signal to a process or a group of processes.
- The `raise` function allows a process to send a signal to itself.
 - The `raise` function was originally defined by ISO C. POSIX.1 includes it to align itself with the ISO C standard, but POSIX.1 extends the specification of `raise` to deal with threads. Since ISO C does not deal with multiple processes, it could not define a function, such as `kill`, that requires a process ID argument.

```
#include <signal.h>

int kill(pid_t pid, int signo);
int raise(int signo);

/* Both return: 0 if OK, -1 on error */
```

The call:

```
raise(signo);
```

is equivalent to the call:

```
kill(getpid(), signo);
```

There are four different conditions for the *pid* argument to kill:

- `pid > 0`. The signal is sent to the process whose process ID is `pid`.
- `pid == 0`. The signal is sent to all processes whose process group ID equals the process group ID of the sender and for which the sender has permission to send the signal.
- `pid < 0`. The signal is sent to all processes whose process group ID equals the absolute value of `pid` and for which the sender has permission to send the signal.
- `pid == -1`. The signal is sent to all processes on the system for which the sender has permission to send the signal.

Note that the term *all processes* in the four conditions above excludes an implementation-defined set of system processes, including kernel processes and `init` (`pid 1`).

A process needs permission to send a signal to another process:

- The superuser can send a signal to any process.
- For other users, the real or effective user ID of the sender has to equal the real or effective user ID of the receiver. If the implementation supports `_POSIX_SAVED_IDS`, the saved set-user-ID of the receiver is checked instead of its effective user ID.
- One special case for the permission testing also exists: if the signal being sent is `SIGCONT`, a process can send it to any other process in the same session.

Some other notes on the `kill` function:

- POSIX.1 defines signal number 0 as the null signal. If the *signo* argument is 0, then the normal error checking is performed by `kill`, but no signal is sent. This technique is often used to determine if a specific process still exists. If we send the process the null signal and it doesn't exist, `kill` returns `-1` and `errno` is set to `ESRCH`. Be aware, however, that UNIX systems recycle process IDs after some amount of time, so the existence of a process with a given process ID does not necessarily mean that it's the process that you think it is.
- The test for process existence is not atomic. By the time that `kill` returns the answer to the caller, the process in question might have exited, so the answer is of limited value.
- If the call to `kill` causes the signal to be generated for the calling process and if the signal is not blocked, either *signo* or some other pending, unblocked signal is delivered to the process before `kill` returns. For additional conditions that occur with threads, see Section 12.8 (`./ch12/#threads-and-signals`).

alarm and pause Functions

The `alarm` function sets a timer that will expire at a specified time in the future. When the timer expires, the `SIGALRM` signal is generated. If we ignore or don't catch this signal, its default action is to terminate the process.

```
#include <unistd.h>

unsigned int alarm(unsigned int seconds);

/* Returns: 0 or number of seconds until previously set alarm */
```

- The *seconds* value is the number of clock seconds in the future when the signal should be generated. When that time occurs, the signal is generated by the kernel, although additional time could elapse before the process gets control to handle the signal, because of processor scheduling delays.
- There is only one of these alarm clocks per process. If `alarm` is called with a previously registered alarm clock not yet expired, then:
 - The number of seconds left for previous alarm clock is returned as the value of this function.
 - The previous alarm clock is replaced by the new value.
- If a previously registered alarm clock for the process has not yet expired and if the *seconds* value is 0, the previous alarm clock is canceled. The number of seconds left for that previous alarm clock is still returned as the value of the function.

Although the default action for `SIGALRM` is to terminate the process, most processes that use an alarm clock catch this signal, which can perform whatever cleanup is required before terminating if the process wants to terminate. If we intend to catch `SIGALRM`, we need to be careful to install its signal handler before calling `alarm`. If we call `alarm` first and are sent `SIGALRM` before we can install the signal handler, our process will terminate.

The `pause` function suspends the calling process until a signal is caught.

```
#include <unistd.h>

int pause(void);

/* Returns: -1 with errno set to EINTR */
```

The only time `pause` returns is if a signal handler is executed and that handler returns. In that case, `pause` returns `-1` with `errno` set to `EINTR`.

sleep1 example

Using `alarm` and `pause`, we can put a process to sleep for a specified amount of time. The following implementation of `sleep1` is incomplete and has problems:

signals/sleep1.c (<https://github.com/shichao-an/apue.3e/blob/master/signals/sleep1.c>)


```

#include    <signal.h>
#include    <unistd.h>

static void
sig_alm(int signo)
{
    /* nothing to do, just return to wake up the pause */
}

unsigned int
sleep1(unsigned int seconds)
{
    if (signal(SIGALRM, sig_alm) == SIG_ERR)
        return(seconds);
    alarm(seconds);    /* start the timer */
    pause();            /* next caught signal wakes us up */
    return(alarm(0));   /* turn off timer, return unslept time */
}

```

This simple implementation has three problems:

1. **If the caller already has an alarm set, that alarm is erased by the first call to `alarm`** . We can correct this by looking at `alarm` 's return value:
 - If the number of seconds until some previously set alarm is less than the argument, then we should wait only until the existing alarm expires.
 - If the previously set alarm will go off after ours, then before returning we should reset this alarm to occur at its designated time in the future.
2. **We have modified the disposition for `SIGALRM`** . If we're writing a function for others to call, we should save the disposition when our function is called and restore it when we're done. We can correct this by saving the return value from `signal` and resetting the disposition before our function returns.
3. **There is a race condition between the first call to `alarm` and the call to `pause`** . On a busy system, it's possible for the alarm to go off and the signal handler to be called before we call `pause` . If that happens, the caller is suspended forever in the call to `pause` (assuming that some other signal isn't caught).

sleep2 example: using `setjmp` and `longjmp`

The following example corrects problem 3 as described above using `setjmp` . The problem 3 can also be corrected using `sigprocmask` and `sigsuspend` , as described in Section 10.19.

signals/sleep2.c (<https://github.com/shichao-an/apue.3e/blob/master/signals/sleep2.c>)

```
#include <setjmp.h>
#include <signal.h>
#include <unistd.h>

static jmp_buf env_alm;

static void
sig_alm(int signo)
{
    longjmp(env_alm, 1);
}

unsigned int
sleep2(unsigned int seconds)
{
    if (signal(SIGALRM, sig_alm) == SIG_ERR)
        return(seconds);
    if (setjmp(env_alm) == 0) {
        alarm(seconds);    /* start the timer */
        pause();           /* next caught signal wakes us up */
    }
    return(alarm(0));      /* turn off timer, return unslept time */
}
```

The `sleep2` function avoids the race condition. Even if the `pause` is never executed, the `sleep2` function returns when the `SIGALRM` occurs.

`sleep2` 's interaction with other signals

There is another subtle problem with the `sleep2` function involving its interaction with other signals. If the `SIGALRM` interrupts some other signal handler, then when we call `longjmp`, we abort the other signal handler, as shown in the following code:

`signals/tsleep2.c` (<https://github.com/shichao-an/apue.3e/blob/master/signals/tsleep2.c>)

```

#include "apue.h"

unsigned int    sleep2(unsigned int);
static void    sig_int(int);

int
main(void)
{
    unsigned int    unslept;

    if (signal(SIGINT, sig_int) == SIG_ERR)
        err_sys("signal(SIGINT) error");
    unslept = sleep2(5);
    printf("sleep2 returned: %u\n", unslept);
    exit(0);
}

static void
sig_int(int signo)
{
    int            i, j;
    volatile int    k;
    /*
     * Tune these loops to run for more than 5 seconds
     * on whatever system this test program is run.
     */
    printf("\nsig_int starting\n");
    for (i = 0; i < 300000; i++)
        for (j = 0; j < 4000; j++)
            k += i * j;
    printf("sig_int finished\n");
}

```

The loop in the `SIGINT` handler was written so that it executes for longer than 5 seconds on one of the systems used by the author. We simply want it to execute longer than the argument to `sleep2`. The integer `k` is declared as `volatile` to prevent an optimizing compiler from discarding the loop.

Run this program and interrupt the sleep by typing the interrupt character:

```

$ ./a.out
^C           # we type the interrupt character
sig_int starting
sleep2 returned: 0

```

The `longjmp` from the `sleep2` function aborted the other signal handler, `sig_int`, even though it wasn't finished.

The above examples of `sleep1` and `sleep2` show the pitfalls in dealing naively with signals. The following sections will show ways around all these problems, so we can handle signals reliably, without interfering with other pieces of code.

Implementing a timeout using `alarm`

A common use for `alarm`, in addition to implementing the `sleep` function, is to put an upper time limit on operations that can block. For example, if we have a `read` operation on a device that can block (slow device, as described in Section 10.5), we might want the `read` to time out after some amount of time. The following example reads one line from standard input (with a timeout) and writes it to standard.

`signals/read1.c` (<https://github.com/shichao-an/apue.3e/blob/master/signals/read1.c>)

```
#include "apue.h"

static void sig_almr(int);

int
main(void)
{
    int      n;
    char     line[MAXLINE];

    if (signal(SIGALRM, sig_almr) == SIG_ERR)
        err_sys("signal(SIGALRM) error");

    alarm(10);
    if ((n = read(STDIN_FILENO, line, MAXLINE)) < 0)
        err_sys("read error");
    alarm(0);

    write(STDOUT_FILENO, line, n);
    exit(0);
}

static void
sig_almr(int signo)
{
    /* nothing to do, just return to interrupt the read */
}
```

Though this code is common in UNIX applications, it has two problems:

1. There is a race condition between the first call to `alarm` and the call to `read`, similar to the first `alarm` and `pause` example in early this section. If the kernel blocks the process between these two function calls for longer than the alarm period, the `read` could block forever, though most operations of this type use a long alarm period (a minute or more) making this unlikely.
2. If system calls are automatically restarted, the `read` is not interrupted when the `SIGALRM` signal handler returns. In this case, the timeout does nothing.

Implementing a timeout with `alarm` and `longjmp`

signals/read2.c (<https://github.com/shichao-an/apue.3e/blob/master/signals/read2.c>)

```
#include "apue.h"
#include <setjmp.h>

static void    sig_alm(int);
static jmp_buf env_alm;

int
main(void)
{
    int      n;
    char     line[MAXLINE];

    if (signal(SIGALRM, sig_alm) == SIG_ERR)
        err_sys("signal(SIGALRM) error");
    if (setjmp(env_alm) != 0)
        err_quit("read timeout");

    alarm(10);
    if ((n = read(STDIN_FILENO, line, MAXLINE)) < 0)
        err_sys("read error");
    alarm(0);

    write(STDOUT_FILENO, line, n);
    exit(0);
}

static void
sig_alm(int signo)
{
    longjmp(env_alm, 1);
}
```

This version works as expected, regardless of whether the system restarts interrupted system calls. However, we still have the problem of interactions with other signal handlers, as described previously.

If we want to set a time limit on an I/O operation, we need to use `longjmp`, as shown previously, while recognizing its possible interaction with other signal handlers. Another option is to use the `select` or `poll` functions described in Section 14.4 (`../io-multiplexing`).

Signal Sets

A **signal set** is a data type to represent multiple signals. This data type is used with functions like `sigprocmask` to tell the kernel not to allow any of the signals in the set to occur. As mentioned earlier, the number of different signals can exceed the number of bits in an integer, so in general we can't use an integer to represent the set with one bit per signal.

POSIX.1 defines the data type `sigset_t` to contain a signal set and the following five functions to manipulate signal sets.

```
#include <signal.h>

int sigemptyset(sigset_t *set);
int sigfillset(sigset_t *set);
int sigaddset(sigset_t *set, int signo);
int sigdelset(sigset_t *set, int signo);

/* All four return: 0 if OK, -1 on error */

int sigismember(const sigset_t *set, int signo);
/* Returns: 1 if true, 0 if false, -1 on error */
```

- `sigemptyset` initializes the signal set pointed to by `set` so that all signals are excluded.
- `sigfillset` initializes the signal set so that all signals are included.
- `sigaddset` adds a single signal to an existing set.
- `sigdelset` removes a single signal from a set.

All applications have to call either `sigemptyset` or `sigfillset` once for each signal set, before using the signal set, because we cannot assume that the C initialization for external and static variables (0) corresponds to the implementation of signal sets on a given system.

In all the functions that take a signal set as an argument, we always pass the address of the signal set as the argument.

Implementation of signal sets

If the implementation has fewer signals than bits in an integer, a signal set can be implemented using one bit per signal. This section assumes that an implementation has 31 signals and 32-bit integers. The `sigemptyset` function zeros the integer, and the `sigfillset` function turns on all the bits in the integer. These two functions can be implemented as macros in the `<signal.h>` header:

```
#define sigemptyset(ptr) (*(ptr) = 0)
#define sigfillset(ptr) (*(ptr) = ~(sigset_t)0, 0)
```

Note that sigfillset must return 0, in addition to setting all the bits on in the signal set, so we use C's comma operator, which returns the value after the comma as the value of the expression.

Using this implementation, `sigaddset` turns on a single bit and `sigdelset` turns off a single bit; `sigismember` tests a certain bit. Since no signal is ever numbered 0, we subtract 1 from the signal number to obtain the bit to manipulate.

signals/setops.c (<https://github.com/shichao-an/apue.3e/blob/master/signals/setops.c>)

```

#include    <signal.h>
#include    <errno.h>

/*
 * <signal.h> usually defines NSIG to include signal number 0.
 */
#define SIGBAD(signo)    ((signo) <= 0 || (signo) >= NSIG)

int
sigaddset(sigset_t *set, int signo)
{
    if (SIGBAD(signo)) {
        errno = EINVAL;
        return(-1);
    }
    *set |= 1 << (signo - 1);    /* turn bit on */
    return(0);
}

int
sigdelset(sigset_t *set, int signo)
{
    if (SIGBAD(signo)) {
        errno = EINVAL;
        return(-1);
    }
    *set &= ~(1 << (signo - 1));    /* turn bit off */
    return(0);
}

int
sigismember(const sigset_t *set, int signo)
{
    if (SIGBAD(signo)) {
        errno = EINVAL;
        return(-1);
    }
    return((*set & (1 << (signo - 1))) != 0);
}

```

sigprocmask Function

As discussed in Section 10.8, the signal mask of a process is the set of signals currently blocked from delivery to that process. A process can examine its signal mask, change its signal mask, or perform both operations in one step by calling the following function.

```
#include <signal.h>

int sigprocmask(int how, const sigset_t *restrict set,
                sigset_t *restrict oset);

/* Returns: 0 if OK, -1 on error */
```

- If *oset* is a non-null pointer, the current signal mask for the process is returned through *oset*.
- If *set* is a non-null pointer, the *how* argument indicates how the current signal mask is modified; if *set* is a null pointer, the signal mask of the process is not changed, and *how* is ignored.
- The *how* argument is one in the following table:

<i>how</i>	Description
SIG_BLOCK	The new signal mask for the process is the union of its current signal mask and the signal set pointed to by <i>set</i> . That is, <i>set</i> contains the additional signals that we want to block.
SIG_UNBLOCK	The new signal mask for the process is the intersection of its current signal mask and the complement of the signal set pointed to by <i>set</i> . That is, <i>set</i> contains the signals that we want to unblock.
SIG_SETMASK	The new signal mask for the process is replaced by the value of the signal set pointed to by <i>set</i> .

After calling `sigprocmask`, if any unblocked signals are pending, at least one of these signals is delivered to the process before `sigprocmask` returns.

Note that the `sigprocmask` function is defined only for single-threaded processes. A separate function, discussed in Section 12.8 ([../ch12/#threads-and-signals](#)) is provided to manipulate a thread's signal mask in a multithreaded process.

The following example shows a function that prints the names of the signals in the signal mask of the calling process.

`lib/prmask.c` (<https://github.com/shichao-an/apue.3e/blob/master/lib/prmask.c>)

```

#include "apue.h"
#include <errno.h>

void
pr_mask(const char *str)
{
    sigset_t    sigset;
    int         errno_save;

    errno_save = errno;    /* we can be called by signal handlers */
    if (sigprocmask(0, NULL, &sigset) < 0) {
        err_ret("sigprocmask error");
    } else {
        printf("%s", str);
        if (sigismember(&sigset, SIGINT))
            printf(" SIGINT");
        if (sigismember(&sigset, SIGQUIT))
            printf(" SIGQUIT");
        if (sigismember(&sigset, SIGUSR1))
            printf(" SIGUSR1");
        if (sigismember(&sigset, SIGALRM))
            printf(" SIGALRM");

        /* remaining signals can go here */

        printf("\n");
    }

    errno = errno_save;    /* restore errno */
}

```

sigpending Function

The `sigpending` function returns the set of signals that are blocked from delivery and currently pending for the calling process. The set of signals is returned through the `set` argument.

```

#include <signal.h>

int sigpending(sigset_t *set);

/* Returns: 0 if OK, -1 on error */

```

Example of `sigpending` and other signal features

The example below shows many of the signal features that have been described.

signals/critical.c (<https://github.com/shichao-an/apue.3e/blob/master/signals/critical.c>)

```
#include "apue.h"

static void sig_quit(int);

int
main(void)
{
    sigset_t    newmask, oldmask, pendmask;

    if (signal(SIGQUIT, sig_quit) == SIG_ERR)
        err_sys("can't catch SIGQUIT");

    /*
     * Block SIGQUIT and save current signal mask.
     */
    sigemptyset(&newmask);
    sigaddset(&newmask, SIGQUIT);
    if (sigprocmask(SIG_BLOCK, &newmask, &oldmask) < 0)
        err_sys("SIG_BLOCK error");

    sleep(5);    /* SIGQUIT here will remain pending */

    if (sigpending(&pendmask) < 0)
        err_sys("sigpending error");
    if (sigismember(&pendmask, SIGQUIT))
        printf("\nSIGQUIT pending\n");

    /*
     * Restore signal mask which unblocks SIGQUIT.
     */
    if (sigprocmask(SIG_SETMASK, &oldmask, NULL) < 0)
        err_sys("SIG_SETMASK error");
    printf("SIGQUIT unblocked\n");

    sleep(5);    /* SIGQUIT here will terminate with core file */
    exit(0);
}

static void
sig_quit(int signo)
{
    printf("caught SIGQUIT\n");
    if (signal(SIGQUIT, SIG_DFL) == SIG_ERR)
```

```
err_sys("can't reset SIGQUIT");
}
```

Run this program:

```
$ ./a.out
^\\ # generate signal once (before 5 seconds are up)
SIGQUIT # pending after return from sleep
caught SIGQUIT # in signal handler
SIGQUIT unblocked # after return from sigprocmask
^\\Quit(coredump) # generate signal again
$ ./a.out
^^\\^^\\^^\\^^\\^^\\^^\\ # generate signal 10 times (before 5 seconds are up)
SIGQUIT pending
caught SIGQUIT # signal is generated only once
SIGQUIT unblocked
^\\Quit(coredump) # generate signal again
```

[p349]

Some notes from this example:

- We saved the old mask when we blocked the signal. To unblock the signal, we did a `SIG_SETMASK` of the old mask. Alternatively, we could `SIG_UNBLOCK` only the signal that we had blocked. Be aware, however, if we write a function that can be called by others and if we need to block a signal in our function, we can't use `SIG_UNBLOCK` to unblock the signal. In this case, we have to use `SIG_SETMASK` and restore the signal mask to its prior value, because it's possible that the caller had specifically blocked this signal before calling our function. We'll see an example of this in the `system` function in Section 10.18.
- When we run the program the second time, we generate the quit signal ten times while the process is asleep, yet the signal is delivered only once to the process when it's unblocked. This demonstrates that signals are not queued on this system

sigaction Function

The `sigaction` function allows us to examine or modify (or both) the action associated with a particular signal. This function supersedes the `signal` function from earlier releases of the UNIX System. Indeed, at the end of this section, we show an implementation of `signal` using `sigaction`.

```
#include <signal.h>

int sigaction(int signo, const struct sigaction *restrict act,
              struct sigaction *restrict oact);

/* Returns: 0 if OK, -1 on error */
```

- The argument *signo* is the signal number whose action we are examining or modifying.
- If the *act* pointer is non-null, we are modifying the action.
- If the *oact* pointer is non-null, the system returns the previous action for the signal through the *oact* pointer

This function uses the following structure:

```
struct sigaction {
    void (*sa_handler)(int); /* addr of signal handler, */
                               /* or SIG_IGN, or SIG_DFL */
    sigset_t sa_mask;         /* additional signals to block */
    int sa_flags;             /* signal options, Figure 10.16 */

    /* alternate handler */
    void (*sa_sigaction)(int, siginfo_t *, void *);
};
```

If the `sa_handler` field contains the address of a signal-catching function (rather than `SIG_IGN` or `SIG_DFL`), then the `sa_mask` field specifies a set of signals that are added to the signal mask of the process before the signal-catching function is called. If and when the signal-catching function returns, the signal mask of the process is reset to its previous value. This enables us to block certain signals whenever a signal handler is invoked. The operating system includes the signal being delivered in the signal mask when the handler is invoked. Hence, we are guaranteed that whenever we are processing a given signal, another occurrence of that same signal is blocked until we're finished processing the first occurrence. Additional occurrences of the same signal are usually not queued (Section 10.8). If the signal occurs five times while it is blocked, when we unblock the signal, the signal-handling function for that signal will usually be invoked only one time.

Once we install an action for a given signal, that action remains installed until we explicitly change it by calling `sigaction`. [p350]

The `sa_flags` field of the `act` structure specifies various options for the handling of this signal. The table below details the meaning of these options when set.

The `sa_sigaction` field is an alternative signal handler used when the `SA_SIGINFO` flag is used with `sigaction`. Implementations might use the same storage for both the `sa_sigaction` field and the `sa_handler` field, so applications can use only one of these fields at a time.

Option flags (`sa_flags`) for the handling of each signal:

Option	SUS	FreeBSD	Linux	Mac OS X	Solaris	Description
SA_INTERRUPT			x			System calls interrupted by this signal are not automatically restarted (the XSI default for <code>sigaction</code>). See Section 10.5.
SA_NOCLDSTOP	x	x	x	x	x	If <i>signo</i> is <code>SIGCHLD</code> , do not generate this signal when a child process stops (job control). This signal is still generated, of course, when a child terminates (but see the <code>SA_NOCLDWAIT</code> option below). When the XSI option is supported, <code>SIGCHLD</code> won't be sent when a stopped child continues if this flag is set.
SA_NOCLDWAIT	x	x	x	x	x	If <i>signo</i> is <code>SIGCHLD</code> , this option prevents the system from creating zombie processes when children of the calling process terminate. If it subsequently calls <code>wait</code> , the calling process blocks until all its child processes have terminated and then returns <code>-1</code> with <code>errno</code> set to <code>ECHILD</code> . (Section 10.7)
SA_NODEFER	x	x	x	x	x	When this signal is caught, the signal is not automatically blocked by the system while the signal-catching function executes (unless the signal is also included in <code>sa_mask</code>). Note that this type of operation corresponds to the earlier unreliable signals.
SA_ONSTACK	XSI	x	x	x	x	If an alternative stack has been declared with <code>sigaltstack(2)</code> , this signal is delivered to the process on the alternative stack.

Option	SUS	FreeBSD	Linux	Mac OS X	Solaris	Description
SA_RESETHAND	x	x	x	x	x	The disposition for this signal is reset to SIG_DFL , and the SA_SIGINFO flag is cleared on entry to the signal-catching function. Note that this type of operation corresponds to the earlier unreliable signals. The disposition for the two signals SIGILL and SIGTRAP can't be reset automatically, however. Setting this flag can optionally cause sigaction to behave as if SA_NODEFER is also set.
SA_RESTART	x	x	x	x	x	System calls interrupted by this signal are automatically restarted. (Section 10.5)
SA_SIGINFO	x	x	x	x	x	This option provides additional information to a signal handler: a pointer to a siginfo structure and a pointer to an identifier for the process context.

Normally, the signal handler is called as:

```
void handler(int signo);
```

If the SA_SIGINFO flag is set, the signal handler is called as:

```
void handler(int signo, siginfo_t *info, void *context);
```

The siginfo structure contains information about why the signal was generated. All POSIX.1-compliant implementations must include at least the si_signo and si_code members. Additionally, implementations that are XSI compliant contain at least the following fields:


```
struct siginfo {
    int si_signo; /* signal number */
    int si_errno; /* if nonzero, errno value from errno.h */
    int si_code; /* additional info (depends on signal) */
    pid_t si_pid; /* sending process ID */
    uid_t si_uid; /* sending process real user ID */
    void *si_addr; /* address that caused the fault */
    int si_status; /* exit value or signal number */
    union signal si_value; /* application-specific value */
    /* possibly other fields also */
};
```

The `siginfo` union contains the following fields:

```
int sival_int;
void *sival_ptr;
```

Applications pass an integer value in `si_value.sival_int` or pass a pointer value in `si_value.sival_ptr` when delivering signals.

If the signal is `SIGCHLD`, then the `si_pid`, `si_status`, and `si_uid` fields will be set. If the signal is `SIGBUS`, `SIGILL`, `SIGFPE`, or `SIGSEGV`, then the `si_addr` contains the address responsible for the fault. The `si_errno` field contains the error number corresponding to the condition that caused the signal to be generated.

The table shows values of `si_code` for various signals, as defined by the Single UNIX Specification:

Signal	Code	Reason
SIGILL	ILL_ILLOPC	illegal opcode
	ILL_ILLOPN	illegal operand
	ILL_ILLADR	illegal addressing mode
	ILL_ILLTRP	illegal trap
	ILL_PRVOPC	privileged opcode
	ILL_PRVREG	privileged register
	ILL_COPROC	coprocessor error
	ILL_BADSTK	internal stack error
SIGFPE	FPE_INTDIV	integer divide by zero
	FPE_INTOVF	integer overflow
	FPE_FLTDIV	floating-point divide by zero
	FPE_FLTOVF	floating-point overflow
	FPE_FLTUND	floating-point underflow
	FPE_FLTRES	floating-point inexact result
	FPE_FLTINV	invalid floating-point operation
	FPE_FLTSUB	subscript out of range
SIGSEGV	SEGV_MAPERR	address not mapped to object
	SEGV_ACCERR	invalid permissions for mapped object
SIGBUS	BUS_ADRALN	invalid address alignment
	BUS_ADRERR	nonexistent physical address
	BUS_OBJERR	object-specific hardware error
SIGTRAP	TRAP_BRKPT	process breakpoint trap
	TRAP_TRACE	process trace trap
SIGCHLD	CLD_EXITED	child has exited
	CLD_KILLED	child has terminated abnormally (no core)
	CLD_DUMPED	child has terminated abnormally with core
	CLD_TRAPPED	traced child has trapped
	CLD_STOPPED	child has stopped
	CLD_CONTINUED	stopped child has continued
Any	SI_USER	signal sent by kill
	SI_QUEUE	signal sent by sigqueue
	SI_TIMER	expiration of a timer set by timer_settime
	SI_ASYNCIO	completion of asynchronous I/O request
	SI_MSGQ	arrival of a message on a message queue (real-time extension)

(../figure_10.17.png)

The *context* argument to the signal handler is a typeless pointer that can be cast to a `ucontext_t` structure identifying the process context at the time of signal delivery. This structure contains at least the following fields:

```
ucontext_t *uc_link;    /* pointer to context resumed when */
                        /* this context returns */
sigset_t uc_sigmask;    /* signals blocked when this context */
                        /* is active */
stack_t uc_stack;       /* stack used by this context */
mcontext_t uc_mcontext; /* machine-specific representation of */
                        /* saved context */
```

The `uc_stack` field describes the stack used by the current context. It contains at least the following members:

```
void *ss_sp; /* stack base or pointer */
size_t ss_size; /* stack size */
int ss_flags; /* flags */
```

When an implementation supports the real-time signal extensions, signal handlers established with the `SA_SIGINFO` flag will result in signals being queued reliably. A separate range of reserved signal numbers is available for real-time application use. Applications can pass information along with the signal by using the `sigqueue` function (Section 10.20 (../sigqueue-function)).

Example: signal Function

The following is the implementation of the `signal` function using `sigaction`. [p354]

lib/signal.c (<https://github.com/shichao-an/apue.3e/blob/master/lib/signal.c>)

```
#include "apue.h"

/* Reliable version of signal(), using POSIX sigaction(). */
Sigfunc *
signal(int signo, Sigfunc *func)
{
    struct sigaction    act, oact;

    act.sa_handler = func;
    sigemptyset(&act.sa_mask);
    act.sa_flags = 0;
    if (signo == SIGALRM) {
#ifdef SA_INTERRUPT
        act.sa_flags |= SA_INTERRUPT;
#endif
    } else {
        act.sa_flags |= SA_RESTART;
    }
    if (sigaction(signo, &act, &oact) < 0)
        return(SIG_ERR);
    return(oact.sa_handler);
}
```

- We must use `sigemptyset` to initialize the `sa_mask` member of the structure since we're not guaranteed that `act.sa_mask = 0` does the same thing.
- We intentionally set the `SA_RESTART` flag for all signals other than `SIGALRM`, so that any system call interrupted by these other signals will be automatically restarted. The reason we don't want `SIGALRM` restarted is to allow us to set a timeout for I/O operations. (Figure 10.10)

- Some older systems, such as SunOS, define the `SA_INTERRUPT` flag. These systems restart interrupted system calls by default, so specifying this flag causes system calls to be interrupted. Linux defines the `SA_INTERRUPT` flag for compatibility with applications that use it, but by default does not restart system calls when the signal handler is installed with `sigaction`. The Single UNIX Specification specifies that the `sigaction` function not restart interrupted system calls unless the `SA_RESTART` flag is specified.

Example: `signal_intr` Function

The following code shows a version of the `signal` function that tries to prevent any interrupted system calls from being restarted.

`lib/signalintr.c` (<https://github.com/shichao-an/apue.3e/blob/master/lib/signalintr.c>)

```
#include "apue.h"

Sigfunc *
signal_intr(int signo, Sigfunc *func)
{
    struct sigaction    act, oact;

    act.sa_handler = func;
    sigemptyset(&act.sa_mask);
    act.sa_flags = 0;
#ifdef SA_INTERRUPT
    act.sa_flags |= SA_INTERRUPT;
#endif
    if (sigaction(signo, &act, &oact) < 0)
        return(SIG_ERR);
    return(oact.sa_handler);
}
```

For improved portability, we specify the `SA_INTERRUPT` flag, if defined by the system, to prevent interrupted system calls from being restarted.

`sigsetjmp` and `siglongjmp` Functions

`setjmp` and `longjmp` functions (Section 7.10 ([../ch7/#setjmp-and-longjmp-functions](#))) can be used for nonlocal branching. The `longjmp` function is often called from a signal handler to return to the main loop of a program, instead of returning from the handler. (Figure 10.8 and Figure 10.11).

However, there is a problem in calling `longjmp`. When a signal is caught, the signal-catching function is entered, with the current signal automatically being added to the signal mask of the process. This prevents subsequent occurrences of that signal from interrupting the signal handler. If we `longjmp` out of the signal handler, what happens to the signal mask for the process depends on the platform:

- Under FreeBSD 8.0 and Mac OS X 10.6.8, `setjmp` and `longjmp` save and restore the signal mask.
- Linux 3.2.0 and Solaris 10, however, do not save and restore the signal mask, although Linux supports an option to provide BSD behavior.
- FreeBSD and Mac OS X provide the functions `_setjmp` and `_longjmp`, which do not save and restore the signal mask.

To allow either form of behavior, POSIX.1 does not specify the effect of `setjmp` and `longjmp` on signal masks. Instead, two new functions, `sigsetjmp` and `siglongjmp`, are defined by POSIX.1. These two functions should always be used when branching from a signal handler.

```
#include <setjmp.h>

int sigsetjmp(sigjmp_buf env, int savemask);
/* Returns: 0 if called directly, nonzero if returning from a call to siglongjmp */

void siglongjmp(sigjmp_buf env, int val);
```

The only difference between these functions and the `setjmp` and `longjmp` functions is that `sigsetjmp` has an additional argument. If *savemask* is nonzero, then `sigsetjmp` also saves the current signal mask of the process in *env*. When `siglongjmp` is called, if the *env* argument was saved by a call to `sigsetjmp` with a nonzero *savemask*, then `siglongjmp` restores the saved signal mask.

signals/mask.c (<https://github.com/shichao-an/apue.3e/blob/master/signals/mask.c>)

```
#include "apue.h"
#include <setjmp.h>
#include <time.h>

static void          sig_usr1(int);
static void          sig_alrm(int);
static sigjmp_buf    jmpbuf;
static volatile sig_atomic_t  canjump;

int
main(void)
{
    if (signal(SIGUSR1, sig_usr1) == SIG_ERR)
        err_sys("signal(SIGUSR1) error");
    if (signal(SIGALRM, sig_alrm) == SIG_ERR)
        err_sys("signal(SIGALRM) error");

    pr_mask("starting main: ");    /* {Prog prmask} */

    if (sigsetjmp(jmpbuf, 1)) {

        pr_mask("ending main: ");

        exit(0);
    }
    canjump = 1;    /* now sigsetjmp() is OK */

    for ( ; ; )
        pause();
}

static void
sig_usr1(int signo)
{
    time_t  starttime;

    if (canjump == 0)
        return;    /* unexpected signal, ignore */

    pr_mask("starting sig_usr1: ");

    alarm(3);    /* SIGALRM in 3 seconds */
    starttime = time(NULL);
    for ( ; ; )    /* busy wait for 5 seconds */
```

```

        if (time(NULL) > starttime + 5)
            break;

    pr_mask("finishing sig_usr1: ");

    canjump = 0;
    siglongjmp(jmpbuf, 1); /* jump back to main, don't return */
}

static void
sig_alrm(int signo)
{
    pr_mask("in sig_alrm: ");
}

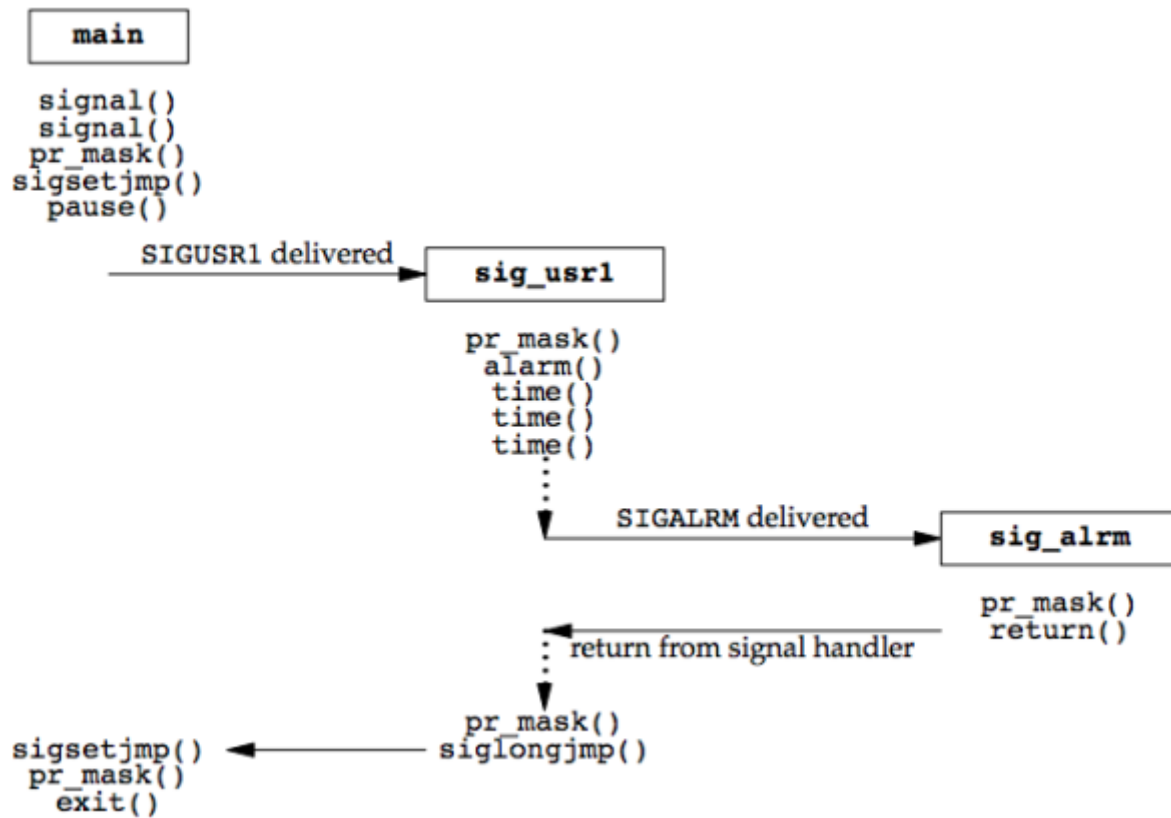
```

- We set the variable `canjump` to a nonzero value only after we've called `sigsetjmp`. This variable is examined in the signal handler, and `siglongjmp` is called only if the flag `canjump` is nonzero. This technique provides protection against the signal handler being called at some earlier or later time, when the jump buffer hasn't been initialized by `sigsetjmp`. This technique should be used whenever `siglongjmp` is called from a signal handler, but is not required with `longjmp` in normal C code. Since a signal can occur at any time, therefore we need the added protection in a signal handler.
- We use the data type `sig_atomic_t`, which is defined by the ISO C standard to be the type of variable that can be written without being interrupted.
 - This means that a variable of type `sig_atomic_t` should not extend across page boundaries on a system with virtual memory and can be accessed with a single machine instruction, for example.
 - We always include the ISO type qualifier `volatile` for these data types as well, since the variable is being accessed by two different threads of control: the `main` function and the asynchronously executing signal handler.

We can divide the following figure into three parts:

- Left part (corresponding to `main`)
- Center part (`sig_usr1`)
- Right part (`sig_alrm`).

While the process is executing in the left part, its signal mask is 0 (no signals are blocked). While executing in the center part, its signal mask is `SIGUSR1`. While executing in the right part, its signal mask is `SIGUSR1 | SIGALRM`.



(../figure_10.21.png)

The output of the program:

```

$ ./a.out &                # start process in background
starting main:
[1] 531                    # the job-control shell prints its process ID
$ kill -USR1 531           # send the process SIGUSR1
starting sig_usr1: SIGUSR1
$ in sig_alm: SIGUSR1 SIGALRM
finishing sig_usr1: SIGUSR1
ending main:
                                # just press RETURN
[1] + Done ./a.out &
  
```

The output is what we expect: when a signal handler is invoked, the signal being caught is added to the current signal mask of the process. The original mask is restored when the signal handler returns. Also, `siglongjmp` restores the signal mask that was saved by `sigsetjmp`.

sigsuspend Function

We have seen how we can change the signal mask for a process to block and unblock selected signals. We can use this technique to protect critical regions of code that we don't want interrupted by a signal. But what if we want to unblock a signal and then `pause`, waiting for the previously blocked signal to occur? Assuming that the signal is `SIGINT`, the incorrect way to do this is:

```
sigset_t newmask, oldmask;

sigemptyset(&newmask);
sigaddset(&newmask, SIGINT);

/* block SIGINT and save current signal mask */
if (sigprocmask(SIG_BLOCK, &newmask, &oldmask) < 0)
    err_sys("SIG_BLOCK error");

/* critical region of code */

/* restore signal mask, which unblocks SIGINT */
if (sigprocmask(SIG_SETMASK, &oldmask, NULL) < 0)
    err_sys("SIG_SETMASK error");

/* window is open */
pause(); /* wait for signal to occur */

/* continue processing */
```

If the signal is sent to the process while it is blocked, the signal delivery will be deferred until the signal is unblocked. To the application, this can look as if the signal occurs between the unblocking and the `pause` (depending on how the kernel implements signals). If this happens, or if the signal does occur between the unblocking and the `pause`, we have a problem. Any occurrence of the signal in this window of time is lost, in the sense that we might not see the signal again, in which case the `pause` will block indefinitely. This is another problem with the earlier unreliable signals.

To correct this problem, we need a way to both restore the signal mask and put the process to sleep in a single atomic operation. This feature is provided by the `sigsuspend` function.

```
#include <signal.h>

int sigsuspend(const sigset_t *sigmask);

/* Returns: -1 with errno set to EINTR */
```

The signal mask of the process is set to the value pointed to by *sigmask*. Then the process is suspended until a signal is caught or until a signal occurs that terminates the process. If a signal is caught and if the signal handler returns, then `sigsuspend` returns, and the signal mask of the process is set to its value before the call to

`sigsuspend` .

Example of `sigsuspend` to protect a critical region

The following code shows the correct way to protect a critical region of code from a specific signal.

`signals/suspend1.c` (<https://github.com/shichao-an/apue.3e/blob/master/signals/suspend1.c>)

```
#include "apue.h"

static void sig_int(int);

int
main(void)
{
    sigset_t    newmask, oldmask, waitmask;

    pr_mask("program start: ");

    if (signal(SIGINT, sig_int) == SIG_ERR)
        err_sys("signal(SIGINT) error");
    sigemptyset(&waitmask);
    sigaddset(&waitmask, SIGUSR1);
    sigemptyset(&newmask);
    sigaddset(&newmask, SIGINT);

    /*
     * Block SIGINT and save current signal mask.
     */
    if (sigprocmask(SIG_BLOCK, &newmask, &oldmask) < 0)
        err_sys("SIG_BLOCK error");

    /*
     * Critical region of code.
     */
    pr_mask("in critical region: ");

    /*
     * Pause, allowing all signals except SIGUSR1.
     */
    if (sigsuspend(&waitmask) != -1)
        err_sys("sigsuspend error");

    pr_mask("after return from sigsuspend: ");

    /*
     * Reset signal mask which unblocks SIGINT.
     */
    if (sigprocmask(SIG_SETMASK, &oldmask, NULL) < 0)
        err_sys("SIG_SETMASK error");

    /*
```

```

    * And continue processing ...
    */
    pr_mask("program exit: ");

    exit(0);
}

static void
sig_int(int signo)
{
    pr_mask("\nin sig_int: ");
}

```

When `sigsuspend` returns, it sets the signal mask to its value before the call. Since `SIGINT` signal will be blocked, so we restore the signal mask to the value that we saved earlier (`oldmask`).

Running the program produces the following output:

```

$ ./a.out
program start:
in critical region: SIGINT
^C                # type the interrupt character
in sig_int: SIGINT SIGUSR1
after return from sigsuspend: SIGINT
program exit:

```

Example of `sigsuspend` to wait for a signal handler to set a global variable

In the following program, we catch both the interrupt signal and the quit signal, but want to wake up the main routine only when the quit signal is caught.

signals/suspend2.c (<https://github.com/shichao-an/apue.3e/blob/master/signals/suspend2.c>)

```
#include "apue.h"

volatile sig_atomic_t  quitflag;  /* set nonzero by signal handler */

static void
sig_int(int signo) /* one signal handler for SIGINT and SIGQUIT */
{
    if (signo == SIGINT)
        printf("\ninterrupt\n");
    else if (signo == SIGQUIT)
        quitflag = 1;  /* set flag for main loop */
}

int
main(void)
{
    sigset_t    newmask, oldmask, zeromask;

    if (signal(SIGINT, sig_int) == SIG_ERR)
        err_sys("signal(SIGINT) error");
    if (signal(SIGQUIT, sig_int) == SIG_ERR)
        err_sys("signal(SIGQUIT) error");

    sigemptyset(&zeromask);
    sigemptyset(&newmask);
    sigaddset(&newmask, SIGQUIT);

    /*
     * Block SIGQUIT and save current signal mask.
     */
    if (sigprocmask(SIG_BLOCK, &newmask, &oldmask) < 0)
        err_sys("SIG_BLOCK error");

    while (quitflag == 0)
        sigsuspend(&zeromask);

    /*
     * SIGQUIT has been caught and is now blocked; do whatever.
     */
    quitflag = 0;

    /*
     * Reset signal mask which unblocks SIGQUIT.
     */
}
```

```
    if (sigprocmask(SIG_SETMASK, &oldmask, NULL) < 0)
        err_sys("SIG_SETMASK error");

    exit(0);
}
```

Sample output from this program is:

```
$ ./a.out
^C          # type the interrupt character
interrupt
^C          # type the interrupt character again
interrupt
^C          # and again
interrupt
^\\ $       # now terminate with the quit character
```

Example of signals that synchronize a parent and child

This example shows how signals can be used to synchronize a parent and child. The following example shows implementations of the five routines `TELL_WAIT`, `TELL_PARENT`, `TELL_CHILD`, `WAIT_PARENT`, and `WAIT_CHILD` from Section 8.9 ([../ch8/#race-conditions](https://github.com/shichao-an/apue.3e/blob/master/lib/tellwait.c)).

[lib/tellwait.c](https://github.com/shichao-an/apue.3e/blob/master/lib/tellwait.c) (<https://github.com/shichao-an/apue.3e/blob/master/lib/tellwait.c>)

```
#include "apue.h"

static volatile sig_atomic_t sigflag; /* set nonzero by sig handler */
static sigset_t newmask, oldmask, zeromask;

static void
sig_usr(int signo) /* one signal handler for SIGUSR1 and SIGUSR2 */
{
    sigflag = 1;
}

void
TELL_WAIT(void)
{
    if (signal(SIGUSR1, sig_usr) == SIG_ERR)
        err_sys("signal(SIGUSR1) error");
    if (signal(SIGUSR2, sig_usr) == SIG_ERR)
        err_sys("signal(SIGUSR2) error");
    sigemptyset(&zeromask);
    sigemptyset(&newmask);
    sigaddset(&newmask, SIGUSR1);
    sigaddset(&newmask, SIGUSR2);

    /* Block SIGUSR1 and SIGUSR2, and save current signal mask */
    if (sigprocmask(SIG_BLOCK, &newmask, &oldmask) < 0)
        err_sys("SIG_BLOCK error");
}

void
TELL_PARENT(pid_t pid)
{
    kill(pid, SIGUSR2); /* tell parent we're done */
}

void
WAIT_PARENT(void)
{
    while (sigflag == 0)
        sigsuspend(&zeromask); /* and wait for parent */
    sigflag = 0;

    /* Reset signal mask to original value */
    if (sigprocmask(SIG_SETMASK, &oldmask, NULL) < 0)
        err_sys("SIG_SETMASK error");
}
```

```
}

void
TELL_CHILD(pid_t pid)
{
    kill(pid, SIGUSR1);          /* tell child we're done */
}

void
WAIT_CHILD(void)
{
    while (sigflag == 0)
        sigsuspend(&zeromask); /* and wait for child */
    sigflag = 0;

    /* Reset signal mask to original value */
    if (sigprocmask(SIG_SETMASK, &oldmask, NULL) < 0)
        err_sys("SIG_SETMASK error");
}
```

In the example, two user-defined signals are used: `SIGUSR1` is sent by the parent to the child, and `SIGUSR2` is sent by the child to the parent.

The `sigsuspend` function is fine if we want to go to sleep while we're waiting for a signal to occur. If we want to call other system functions while we're waiting, the only solution is to use multiple threads and dedicate a separate thread to handling signals (Section 12.8 (../ch12/#threads-and-signals)).

Without using threads, the best we can do is to set a global variable in the signal handler when the signal occurs. For example, if we catch both `SIGINT` and `SIGALRM` and install the signal handlers using the `signal_intr` function, the signals will interrupt any slow system call that is blocked. The signals are most likely to occur when we're blocked in a call to the `read` function waiting for input from a slow device. (This is especially true for `SIGALRM`, since we set the alarm clock to prevent us from waiting forever for input.) The code to handle this looks similar to the following:


```

if (intr_flag)      /* flag set by our SIGINT handler */
    handle_intr();
if (alarm_flag)     /* flag set by our SIGALRM handler */
    handle_alarm();

/* signals occurring in here are lost */

while (read( ... ) < 0) {
    if (errno == EINTR) {
        if (alarm_flag)
            handle_alarm();
        else if (intr_flag)
            handle_intr();
    } else {
        /* some other error */
    }
} else if (n == 0) {
    /* end of file */
} else {
    /* process input */
}

```

We test each of the global flags before calling `read` and again if `read` returns an interrupted system call error. The problem occurs if either signal is caught between the first two if statements and the subsequent call to `read`. Signals occurring in here are lost, as indicated by the code comment. The signal handlers are called, and they set the appropriate global variable, but the `read` never returns (unless some data is ready to be read).

What we would like to be able to do is the following sequence of steps, in order.

1. Block `SIGINT` and `SIGALRM`.
2. Test the two global variables to see whether either signal has occurred and, if so, handle the condition.
3. Call `read` (or any other system function) and unblock the two signals, as an atomic operation.

The `sigsuspend` function helps us only if step 3 is a `pause` operation.

abort Function

The `abort` function causes abnormal program termination.

```

#include <stdlib.h>

void abort(void);

/* This function never returns */

```

The `abort` function sends the `SIGABRT` signal to the caller. Processes should not ignore this signal. ISO C states that calling `abort` will deliver an unsuccessful termination notification to the host environment by calling `raise(SIGABRT)`.

[p365]

ISO C:

- If the signal is caught and the signal handler returns, `abort` still doesn't return to its caller. If this signal is caught, the only way the signal handler can't return is if it calls `exit`, `_exit`, `_Exit`, `longjmp`, or `siglongjmp`.
- It is up to the implementation as to whether output streams are flushed and whether temporary files (Section 5.13 (../ch5/#temporary-files)) are deleted.

POSIX.1:

- `abort` overrides the blocking or ignoring of the signal by the process.
- The intent of letting the process catch the `SIGABRT` is to allow it to perform any cleanup that it wants to do before the process terminates.
- If the process doesn't terminate itself from this signal handler, when the signal handler returns, `abort` terminates the process.
- An implementation is allowed to call `fclose` on open standard I/O streams before terminating if the call to `abort` terminates the process.

Since most UNIX System implementations of `tmpfile` call `unlink` immediately after creating the file, the ISO C warning about temporary files does not usually concern us. [p366]

The following is an implementation of the `abort` function as specified by POSIX.1:

```

#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void
abort(void)          /* POSIX-style abort() function */
{
    sigset_t          mask;
    struct sigaction   action;

    /* Caller can't ignore SIGABRT, if so reset to default */
    sigaction(SIGABRT, NULL, &action);
    if (action.sa_handler == SIG_IGN) {
        action.sa_handler = SIG_DFL;
        sigaction(SIGABRT, &action, NULL);
    }
    if (action.sa_handler == SIG_DFL)
        fflush(NULL);          /* flush all open stdio streams */

    /* Caller can't block SIGABRT; make sure it's unblocked */
    sigfillset(&mask);
    sigdelset(&mask, SIGABRT); /* mask has only SIGABRT turned off */
    sigprocmask(SIG_SETMASK, &mask, NULL);
    kill(getpid(), SIGABRT);    /* send the signal */

    /* If we're here, process caught SIGABRT and returned */
    fflush(NULL);              /* flush all open stdio streams */
    action.sa_handler = SIG_DFL;
    sigaction(SIGABRT, &action, NULL); /* reset to default */
    sigprocmask(SIG_SETMASK, &mask, NULL); /* just in case ... */
    kill(getpid(), SIGABRT);        /* and one more time */
    exit(1);                      /* this should never be executed ... */
}

```

- This implementation of `abort` first check whether the default action will occur; if so, it flush all the standard I/O streams. This is not equivalent to calling `fclose` on all the open streams (since it just flushes them and doesn't close them), but when the process terminates, the system closes all open files.
- If the process catches the signal and returns, we flush all the streams again, since the process could have generated more output.
- The only condition we don't handle is the case where the process catches the signal and calls `_exit` or `_Exit`. In this case, any unflushed standard I/O buffers in memory are discarded. (We assume that a caller that does this doesn't want the buffers flushed.)

- As mentioned in Section 10.9, if calling `kill` causes the signal to be generated for the caller, and if the signal is not blocked, then the signal (or some other pending, unlocked signal) is delivered to the process before `kill` returns. We block all signals except `SIGABRT`, so we know that if the call to `kill` returns, the process caught the signal and the signal handler returned.

system Function

Section 8.13 ([../ch8/##system-function](#)) showed an implementation of the `system` function, which did not do any signal handling. POSIX.1 requires that `system` ignore `SIGINT` and `SIGQUIT` and block `SIGCHLD`. Before showing a version that handles these signals correctly, let's see why we need to worry about signal handling.

Example of `system` invoking `ed` editor

The program (Figure 10.26) shown below uses the `system` function from Section 8.13 ([../ch8/##system-function](#)) to invoke the `ed(1)` editor. It is an interactive program that catches the interrupt and quit signals. If `ed` is invoked from a shell and type the interrupt character, it catches the interrupt signal and prints a question mark. The `ed` program also sets the disposition of the quit signal so that it is ignored.

`signals/systest2.c` (<https://github.com/shichao-an/apue.3e/blob/master/signals/systest2.c>)

```

#include "apue.h"

static void
sig_int(int signo)
{
    printf("caught SIGINT\n");
}

static void
sig_chld(int signo)
{
    printf("caught SIGCHLD\n");
}

int
main(void)
{
    if (signal(SIGINT, sig_int) == SIG_ERR)
        err_sys("signal(SIGINT) error");
    if (signal(SIGCHLD, sig_chld) == SIG_ERR)
        err_sys("signal(SIGCHLD) error");
    if (system("/bin/ed") < 0)
        err_sys("system() error");
    exit(0);
}

```

This program catches both `SIGINT` and `SIGCHLD`. If we invoke the program, we get:

```

$ ./a.out
a                # append text to the editor's buffer
Here is one line of text
.                # period on a line by itself stops append mode
1,$p             # print first through last lines of buffer to see what's
there
Here is one line of text
w temp.foo       # write the buffer to a file
25               # editor says it wrote 25 bytes
q                # and leave the editor
caught SIGCHLD

```

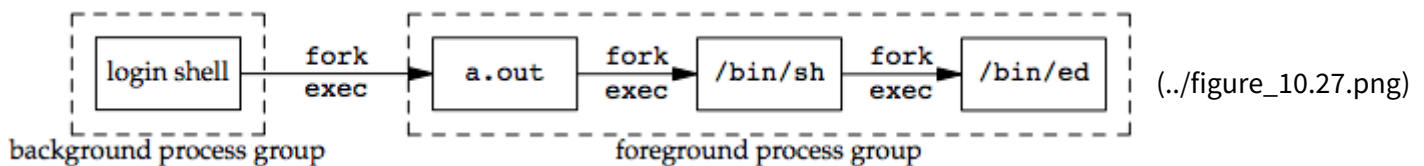
- When the editor terminates, the kernel sends the `SIGCHLD` signal to the parent (the `a.out` process) and it is caught and returned from the signal handler. The parent should be catching the `SIGCHLD` signal because it has created its own children, so that it knows when its children have terminated.

- The delivery of the SIGCHLD signal in the parent should be blocked while the system function is executing, as specified by POSIX.1. Otherwise, when the child created by `system` terminates, it would fool the caller of `system` into thinking that one of its own children terminated. The caller would then use one of the `wait` functions to get the termination status of the child, thereby preventing the `system` function from being able to obtain the child's termination status for its return value.

If we run the program again, this time sending the editor an interrupt signal, we get

```
$ ./a.out
a                # append text to the editor's buffer
hello, world
.                # period on a line by itself stops append mode
1,$p             # print first through last lines to see what's there
hello, world
w temp.foo       # write the buffer to a file
13               # editor says it wrote 13 bytes
^C               # type the interrupt character
?                # editor catches signal, prints question mark
caught SIGINT    # and so does the parent process
q                # leave editor
caught SIGCHLD
```

As mentioned in Section 9.6 (`../ch9/#controlling-terminal`), typing the interrupt character causes the interrupt signal to be sent to all the processes in the foreground process group. The following figure shows the arrangement of the processes when the editor is running.



In this example:

`SIGINT` is sent to all three foreground processes (the shell ignores it) and both the `a.out` process and the editor catch the signal. When running another program with the `system` function, we shouldn't have both the parent and the child catching the two terminal-generated signals: interrupt and quit. Instead, these two signals should be sent to the program that is running: the child. Since the command that is executed by `system` can be an interactive command (the `ed` program in this example) and since the caller of `system` gives up control while the program executes, waiting for it to finish, the caller of `system` should not be receiving these two terminal-generated signals. For this reason, POSIX.1 specifies that the `system` function should ignore the `SIGINT` and `SIGQUIT` signals while waiting for the command to complete.

Implementation of `system` with signal handling

The program below shows an implementation of the `system` function with the required signal handling.

signals/system.c (<https://github.com/shichao-an/apue.3e/blob/master/signals/system.c>)

```
#include <sys/wait.h>
#include <errno.h>
#include <signal.h>
#include <unistd.h>

int
system(const char *cmdstring) /* with appropriate signal handling */
{
    pid_t      pid;
    int        status;
    struct sigaction ignore, saveintr, savequit;
    sigset_t   chldmask, savemask;

    if (cmdstring == NULL)
        return(1); /* always a command processor with UNIX */

    ignore.sa_handler = SIG_IGN; /* ignore SIGINT and SIGQUIT */
    sigemptyset(&ignore.sa_mask);
    ignore.sa_flags = 0;
    if (sigaction(SIGINT, &ignore, &saveintr) < 0)
        return(-1);
    if (sigaction(SIGQUIT, &ignore, &savequit) < 0)
        return(-1);
    sigemptyset(&chldmask); /* now block SIGCHLD */
    sigaddset(&chldmask, SIGCHLD);
    if (sigprocmask(SIG_BLOCK, &chldmask, &savemask) < 0)
        return(-1);

    if ((pid = fork()) < 0) {
        status = -1; /* probably out of processes */
    } else if (pid == 0) { /* child */
        /* restore previous signal actions & reset signal mask */
        sigaction(SIGINT, &saveintr, NULL);
        sigaction(SIGQUIT, &savequit, NULL);
        sigprocmask(SIG_SETMASK, &savemask, NULL);

        execl("/bin/sh", "sh", "-c", cmdstring, (char *)0);
        _exit(127); /* exec error */
    } else { /* parent */
        while (waitpid(pid, &status, 0) < 0)
            if (errno != EINTR) {
                status = -1; /* error other than EINTR from waitpid() */
                break;
            }
    }
}
```



```

    }

    /* restore previous signal actions & reset signal mask */
    if (sigaction(SIGINT, &saveintr, NULL) < 0)
        return(-1);
    if (sigaction(SIGQUIT, &savequit, NULL) < 0)
        return(-1);
    if (sigprocmask(SIG_SETMASK, &savemask, NULL) < 0)
        return(-1);

    return(status);
}

```

This implementation of `system` differs from the previous flawed one in the following ways:

1. No signal is sent to the calling process when we type the interrupt or quit character.
2. When the `ed` command exits, `SIGCHLD` is not sent to the calling process. Instead, it is blocked until we unblock it in the last call to `sigprocmask`, after the `system` function retrieves the child's termination status by calling `waitpid`.

POSIX.1 states that if `wait` or `waitpid` returns the status of a child process while `SIGCHLD` is pending, then `SIGCHLD` should not be delivered to the process unless the status of another child process is also available. FreeBSD 8.0, Mac OS X 10.6.8, and Solaris 10 all implement this semantic, while Linux 3.2.0 doesn't. In Linux, `SIGCHLD` remains pending after the `system` function calls `waitpid`; when the signal is unblocked, it is delivered to the caller. If we called `wait` in the `sig_chld` function in Figure 10.26, a Linux system would return `-1` with `errno` set to `ECHILD`, since the `system` function already retrieved the termination status of the child.

Many older texts show the ignoring of the interrupt and quit signals as follows:

```

if ((pid = fork()) < 0) {
    err_sys("fork error");
} else if (pid == 0) {
    /* child */
    execl(...);
    _exit(127);
}

/* parent */
old_intr = signal(SIGINT, SIG_IGN);
old_quit = signal(SIGQUIT, SIG_IGN);
waitpid(pid, &status, 0)
signal(SIGINT, old_intr);
signal(SIGQUIT, old_quit);

```

The problem with this sequence of code is that we have no guarantee after the `fork` regarding whether the parent or child runs first. If the child runs first and the parent doesn't run for some time after, an interrupt signal might be generated before the parent is able to change its disposition to be ignored. For this reason, in the implementation of `system`, we change the disposition of the signals before the `fork`.

Note that we have to reset the dispositions of these two signals in the child before the call to `exec1`. This allows `exec1` to change their dispositions to the default, based on the caller's dispositions, as described in Section 8.10 ([../ch8/#exec-functions](#)).

Return Value from `system`

The return value from `system` is the termination status of the shell, which isn't always the termination status of the command string. [p371]

Run the program in Figure 8.24 (<https://github.com/shichao-an/apue.3e/blob/master/proc/systest3.c>) and send some signals to the command that's executing:

```
$ tsys "sleep 30"
^Cnormal termination, exit status = 130 # we press the interrupt key
$ tsys "sleep 30"
^\\sh: 946 # Quit we press the quit key
normal termination, exit status = 131
```

When we terminate the `sleep` call with the interrupt signal, the `pr_exit` function (Figure 8.5) thinks that it terminated normally. The same thing happens when we kill the `sleep` call with the quit key. This is because the Bourne shell has a poorly documented feature in which its termination status is 128 plus the signal number, when the command it was executing is terminated by a signal. [p372]

Try a similar example, but this time we'll send a signal directly to the shell and see what is returned by `system`:

```
$ tsys "sleep 30" & # start it in background this time
9257
$ ps -f # look at the process IDs
UID PID PPID TTY TIME CMD
sar 9260 949 pts/5 0:00 ps -f
sar 9258 9257 pts/5 0:00 sh -c sleep 30
sar 949 947 pts/5 0:01 /bin/sh
sar 9257 949 pts/5 0:00 tsys sleep 30
sar 9259 9258 pts/5 0:00 sleep 30
$ kill -KILL 9258 # kill the shell itself
abnormal termination, signal number = 9
```

We can see that the return value from `system` reports an abnormal termination only when the shell itself terminates abnormally.

Other shells behave differently when handling terminal-generated signals, such as `SIGINT` and `SIGQUIT`. With `bash` and `dash`, for example, pressing the interrupt or quit key will result in an exit status indicating abnormal termination with the corresponding signal number. However, if we find our process executing `sleep` and send it a signal directly, so that the signal goes only to the individual process instead of the entire foreground process group, we will find that these shells behave like the Bourne shell and exit with a normal termination status of 128 plus the signal number.

When writing programs that use the `system` function, be sure to interpret the return value correctly. If you call `fork`, `exec`, and `wait` yourself, the termination status is not the same as if you call `system`.

`sleep`, `nanosleep`, and `clock_nanosleep` Functions

```
#include <unistd.h>

unsigned int sleep(unsigned int seconds);

/* Returns: 0 or number of unslept seconds */
```

The `sleep` function causes the calling process to be suspended until either:

1. The amount of wall clock time specified by `seconds` has elapsed. In this case, the return value is 0.
2. A signal is caught by the process and the signal handler returns. In this case the return value is the number of unslept seconds (the requested time minus the actual time slept).

As with an `alarm` signal, the actual return may occur at a time later than requested because of other system activity.

Although `sleep` can be implemented with the `alarm` function (Section 10.10), this isn't required. If `alarm` is used, there can be interactions between the two functions. The POSIX.1 standard leaves all these interactions unspecified.

FreeBSD 8.0, Linux 3.2.0, Mac OS X 10.6.8, and Solaris 10 implement `sleep` using the `nanosleep` function, which allows the implementation to be independent of signals and the alarm timer.

The follow example shows an implementation of the POSIX.1 `sleep` function. This function is a modification of Figure 10.7, which handles signals reliably, avoiding the race condition in the earlier implementation, but does not handle any interactions with previously set alarms.

`lib/sleep.c` (<https://github.com/shichao-an/apue.3e/blob/master/lib/sleep.c>)

```
#include "apue.h"

static void
sig_alm(int signo)
{
    /* nothing to do, just returning wakes up sigsuspend() */
}

unsigned int
sleep(unsigned int seconds)
{
    struct sigaction    newact, oldact;
    sigset_t            newmask, oldmask, suspmask;
    unsigned int        unslept;

    /* set our handler, save previous information */
    newact.sa_handler = sig_alm;
    sigemptyset(&newact.sa_mask);
    newact.sa_flags = 0;
    sigaction(SIGALRM, &newact, &oldact);

    /* block SIGALRM and save current signal mask */
    sigemptyset(&newmask);
    sigaddset(&newmask, SIGALRM);
    sigprocmask(SIG_BLOCK, &newmask, &oldmask);

    alarm(seconds);
    suspmask = oldmask;

    /* make sure SIGALRM isn't blocked */
    sigdelset(&suspmask, SIGALRM);

    /* wait for any signal to be caught */
    sigsuspend(&suspmask);

    /* some signal has been caught, SIGALRM is now blocked */

    unslept = alarm(0);

    /* reset previous action */
    sigaction(SIGALRM, &oldact, NULL);

    /* reset signal mask, which unblocks SIGALRM */
    sigprocmask(SIG_SETMASK, &oldmask, NULL);
}
```

```
    return(unslept);
}
```

This code doesn't use any form of nonlocal branching (as in Figure 10.8 to avoid the race condition between alarm and pause), so there is no effect on other signal handlers that may be executing when the SIGALRM is handled.

The `nanosleep` function is similar to the `sleep` function, but provides nanosecond-level granularity.

```
#include <time.h>

int nanosleep(const struct timespec *reqtp, struct timespec *remtp);

/* Returns: 0 if slept for requested time or -1 on error */
```

This function suspends the calling process until either the requested time has elapsed or the function is interrupted by a signal.

Arguments:

- The *reqtp* parameter specifies the amount of time to sleep in seconds and nanoseconds.
- The *remtp* parameter. If the sleep interval is interrupted by a signal and the process doesn't terminate, the `timespec` structure pointed to by the *remtp* parameter will be set to the amount of time left in the sleep interval. We can set this parameter to NULL if we are uninterested in the time unslept.

Notes on `nanosleep` :

- If the system doesn't support nanosecond granularity, the requested time is rounded up.
- Because the `nanosleep` function doesn't involve the generation of any signals, we can use it without worrying about interactions with other functions.

The `clock_nanosleep` function provides the capability to suspend the calling thread using a delay time relative to a particular clock, using multiple system clocks (Section 6.10 (../ch6/#time-and-date-routines))

```
#include <time.h>

int clock_nanosleep(clockid_t clock_id, int flags,
                    const struct timespec *reqtp, struct timespec *remtp);

/* Returns: 0 if slept for requested time or error number on failure */
```

Arguments:

- The *clock_id* argument specifies the clock against which the time delay is evaluated. Identifiers for clocks are listed in Figure 6.8.
- The *flags* argument is used to control whether the delay is absolute or relative.
 - When *flags* is set to 0, the sleep time is relative (how long we want to sleep).

- When it is set to `TIMER_ABSTIME`, the sleep time is absolute (we want to sleep until the clock reaches the specified time).
- The `reqtp` and `remtp` arguments are the same as in the `nanosleep` function. However:
 - When we use an absolute time, the `remtp` argument is unused, because it isn't needed.
 - We can reuse the same value for the `reqtp` argument for additional calls to `clock_nanosleep` until the clock reaches the specified absolute time value.

Except for error returns, the call:

```
clock_nanosleep(CLOCK_REALTIME, 0, reqtp, remtp);
```

has the same effect as the call:

```
nanosleep(reqtp, remtp);
```

Some applications require precision with how long they sleep, and a relative sleep time can lead to sleeping longer than desired. Using an absolute time improves the precision, even though a time-sharing process scheduler makes no guarantee that our task will execute immediately after our sleep time has ended. [p375]

sigqueue Function

As discussed in Section 10.8, most UNIX systems don't queue signals. With the real-time extensions to POSIX.1, some systems began adding support for queueing signals. With SUSv4, the queued signal functionality has moved from the real-time extensions to the base specification.

These extensions allow applications to pass more information along with the delivery (Section 10.14). This information is embedded in a `siginfo` structure.

To use queued signals we have to do the following:

1. Specify the `SA_SIGINFO` flag when we install a signal handler using the `sigaction` function. Without this flag, it is left up to the implementation whether the signal is queued.
2. Provide a signal handler in the `sa_sigaction` member of the `sigaction` structure instead of using the usual `sa_handler` field. Implementations might allow us to use the `sa_handler` field, but we won't be able to obtain the extra information sent with the `sigqueue` function.
3. Use the `sigqueue` function to send signals.

```
#include <signal.h>

int sigqueue(pid_t pid, int signo, const union sigval value)

/* Returns: 0 if OK, -1 on error */
```

The `sigqueue` function is similar to the `kill` function, except that:

- We can only direct signals to a single process
- We can use the *value* argument to transmit either an integer or a pointer value to the signal handler.

Signals can't be queued infinitely. When the SIGQUEUE_MAX limit (POSIX Limits (../ch2/#posix-limits)) is reached, `sigqueue` can fail with `errno` set to `EAGAIN`.

With the real-time signal enhancements, other signal numbers between `SIGRTMIN` and `SIGRTMAX` inclusive can be queued, and the default action for these signals is to terminate the process. [p376]

The table below summarizes behavior of queued signals on various platforms:

Behavior	SUS	FreeBSD	Linux	Mac OS X	Solaris
supports <code>sigqueue</code>	x	x	x	x	x
queues other signals besides <code>SIGRTMIN</code> to <code>SIGRTMAX</code>	optional	x			x
queues signals even if the caller doesn't use the <code>SA_SIGINFO</code>	optional	x	x		

Job-Control Signals

POSIX.1 considers six to be job-control signals:

<code>SIGCHLD</code>	Child process has stopped or terminated.
<code>SIGCONT</code>	Continue process, if stopped.
<code>SIGSTOP</code>	Stop signal (can't be caught or ignored).
<code>SIGTSTP</code>	Interactive stop signal.
<code>SIGTTIN</code>	Read from controlling terminal by background process group member.
<code>SIGTTOU</code>	Write to controlling terminal by a background process group member.

The following program demonstrates the normal sequence of code used when a program handles job control.

`signals/sigtstp.c` (<https://github.com/shichao-an/apue.3e/blob/master/signals/sigtstp.c>)

```
#include "apue.h"

#define BUFFSIZE    1024

static void
sig_tstp(int signo) /* signal handler for SIGTSTP */
{
    sigset_t    mask;

    /* ... move cursor to lower left corner, reset tty mode ... */

    /*
     * Unblock SIGTSTP, since it's blocked while we're handling it.
     */
    sigemptyset(&mask);
    sigaddset(&mask, SIGTSTP);
    sigprocmask(SIG_UNBLOCK, &mask, NULL);

    signal(SIGTSTP, SIG_DFL); /* reset disposition to default */

    kill(getpid(), SIGTSTP); /* and send the signal to ourself */

    /* we won't return from the kill until we're continued */

    signal(SIGTSTP, sig_tstp); /* reestablish signal handler */

    /* ... reset tty mode, redraw screen ... */
}

int
main(void)
{
    int        n;
    char       buf[BUFFSIZE];

    /*
     * Only catch SIGTSTP if we're running with a job-control shell.
     */
    if (signal(SIGTSTP, SIG_IGN) == SIG_DFL)
        signal(SIGTSTP, sig_tstp);

    while ((n = read(STDIN_FILENO, buf, BUFFSIZE)) > 0)
        if (write(STDOUT_FILENO, buf, n) != n)
            err_sys("write error");
}
```



```

    if (n < 0)
        err_sys("read error");

    exit(0);
}

```

This program does the following:

- When the program starts, it arranges to catch the `SIGTSTP` signal only if the signal's disposition is `SIG_DFL`. The reason is that when the program is started by a shell that doesn't support job control (`/bin/sh` , for example), the signal's disposition should be set to `SIG_IGN` . In fact, the shell doesn't explicitly ignore this signal; `init` sets the disposition of the three job-control signals (`SIGTSTP` , `SIGTTIN` , and `SIGTTOU`) to `SIG_IGN` . This disposition is then inherited by all login shells. Only a job-control shell should reset the disposition of these three signals to `SIG_DFL` .
- When we type the suspend character, the process receives the `SIGTSTP` signal and the signal handler is invoked. At this point, we would do any terminal-related processing: move the cursor to the lower-left corner, restore the terminal mode, etc.
- After resetting the disposition of `SIGSTOP` to its default (stop the process) and unblocking it, we send ourself the `SIGSTOP` signal (`kill(getpid(), SIGTSTP);`). We have to unblock it since we're currently handling that same signal, and the system blocks it automatically while it's being caught.
- At this point, the system stops the process. It is continued only when it receives (usually from the job-control shell, in response to an interactive `fg` command) a `SIGCONT` signal.
- We don't catch `SIGCONT` since its default disposition is to continue the stopped process; when this happens, the program continues as though it returned from the `kill` function. When the program is continued, we reset the disposition for the `SIGTSTP` signal and do any terminal processing (as indicated in the comments).

Signal Names and Numbers

This section discusses how to map between signal numbers and names. Some systems provide the array:

```
extern char *sys_siglist[];
```

The array index is the signal number, giving a pointer to the character string name of the signal. [p379]

To print the signal's character string in a portable manner, use the `psignal` function:

```

#include <signal.h>

void psignal(int signo, const char *msg);

```

The string *msg* (which normally includes the name of the program) is output to the standard error, followed by a colon and a space, followed by a description of the signal, followed by a newline. If *msg* is `NULL`, then only the description is written to the standard error. This function is similar to `perror` (Section 1.7 ([../ch1/#error-handling](#))).

If you have a `siginfo` structure from an alternative `sigaction` signal handler, you can print the signal information with the `psiginfo` function.

```
#include <signal.h>

void psiginfo(const siginfo_t *info, const char *msg);
```

You can use the `strsignal` function if you only need the string description of the signal. This function is similar to `strerror` (Section 1.7 ([../ch1/#error-handling](#))).

```
#include <string.h>

char *strsignal(int signo);

/* Returns: a pointer to a string describing the signal */
```

Summary

An understanding of signal handling is essential to advanced UNIX System programming. This chapter has taken a long and thorough look at UNIX System signals, from previous implementations to the POSIX.1 reliable-signal concept and all the related functions, followed by POSIX.1 `abort` , `system` , and `sleep` functions.

Doubts and Solutions

Verbatim

p324 on some macro constants in `<signal.h>`

These constants can be used in place of the "pointer to a function that takes an integer argument and returns nothing", the second argument to `signal` , and the return value from `signal` . The three values used for these constants need not be `-1`, `0`, and `1`. They must be three values that can never be the address of any declarable function. Most UNIX systems use the values shown.

Question: Why is the macro in the form `SIG_ERR (void (*)())-1` and the like?

Solution: They are integer that cast into an address which means the "pointer to a function that takes an integer argument and returns nothing". `void (*)()` tells the compiler to ignore type-checking for the parameters. See:

- Stack Overflow (<http://stackoverflow.com/questions/22344303>)

p359 on unblocking signals

If the signal is sent to the process while it is blocked, the signal delivery will be deferred until the signal is unblocked. To the application, this can look as if the signal occurs between the unblocking and the **pause** (depending on how the kernel implements signals). If this happens, or if the signal does occur between the unblocking and the **pause**, we have a problem. Any occurrence of the signal in this window of time is lost, in the sense that we might not see the signal again, in which case the **pause** will block indefinitely. This is another problem with the earlier unreliable signals.

Question: What is exactly the window? Shouldn't be the unblocked signals delivered to the process? Not fully understood.